



CS6004NI Application Development

30% Individual Coursework

2025-26 Autumn Credit:

30

Student Name: Simanka Paudel

London Met ID: 23050271

College ID: NP01CP4A230098

Assignment Due Date: Wednesday, January 28, 2026

Assignment Submission Date: Wednesday, January 28, 2026

Module Leader: Mr. Bikram Poudel

Word Count: 5910

Project File Links:

Drive Link:	23050271 Simanka Paudel Development
Git hub link:	https://github.com/SimankaPaudel/JournalApp.git

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked.am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
1.1	Purpose.....	1
1.2	Scope.....	2
1.3	Objectives	2
1.4	Technology Stack.....	3
2	Features and functionalities	5
2.1	Feature Breakdown.....	5
2.2	Description of features with evidence.....	8
2.2.1	CRUD (Create,Read,Update,Delete) for Journal Entries.....	8
2.2.2	Rich Text/ Markdown writing support.....	12
2.2.3	Mood Tracking.....	13
2.2.4	Login / Registration.....	16
2.2.5	Tagging system	18
2.2.6	Calendar Preview	19
2.2.7	Paginated Journal View.....	20
2.2.8	Search and filter	21
2.2.9	Streak Tracking	26
2.2.10	Theme Customization	26
2.2.11	Dashboard Analytics and Insights	28
2.2.12	Security and Privacy.....	29
2.2.13	Exporting Journal Entries as PDF	30
3	Proof of work.....	33
3.1	Design.....	33
3.1.1	Wireframes	33
3.1.2	UI Designs.....	39
3.1.3	Use Case Diagram	44
3.1.4	Entity Relationship Diagram	45
3.1.5	Milestone chart	46
3.2	Vcs repository	47
3.3	Quality.....	49
3.3.1	Code Readability	49

3.3.2	Code Efficiency	53
3.3.3	Code Modularity	54
3.3.4	Error handling	56
3.4	Testing	59
3.4.1	Test 1: Authentication and Access Control	59
3.4.2	Test 3: Journal Entry Mood Selection Validation	61
3.4.3	Test 4: Tag Assignment.....	62
3.4.4	Test 5: Rich Text and Markdown Formatting	64
3.4.5	Test 6: Journal Entry CRUD	65
3.4.6	Test 7: Calendar-Based Entry Navigation.....	70
3.4.7	Test 8: Paginated view of past journal entries	71
3.4.8	Test 9: Search and filter of journals	72
3.4.9	Test 10: Journaling streak tracking	76
3.4.10	Test 11: Exporting entries as pdf.....	77
3.4.11	Test 12: Light mode switch.....	80
4	Individual Reflection	82
5	Conclusion	83
5.1	Future Work	84

Table of figures

Figure 1: Journal page entry load	8
Figure 2: Journal Creation	9
Figure 3: Saved journal preview	10
Figure 4: View journal entry	11
Figure 5: Journal entry update.....	11
Figure 6: Journal update preview	11
Figure 7: Journal deletion	12
Figure 8: screenshot of journal entry markdown feature	12
Figure 9: Screenshot of a journal entry displaying formatted text.	13
Figure 10: Mood dropdown content.....	14
Figure 11: Secondary mood dropdown	15
Figure 12: Mood selection	16
Figure 13: Mood selected shown in journal	16
Figure 14: Registration page load	17
Figure 15: Login page render	17
Figure 16: Tag selection with custom tag	18
Figure 17: Selected tag shown in journal.....	19
Figure 18: Calendar preview	20
Figure 19: Screenshot showing the paginated journal list with navigation controls.	21
Figure 20: Screenshot of search and filter page	22
Figure 21: Screenshot displaying the search bar and applied filter options.....	22
Figure 22: Screenshot displaying the result after searching	23
Figure 23: Screenshot of searching and filtering by date range	23
Figure 24: Screenshot of search and filter result by date range	24
Figure 25: Screenshot of search and filter by mood	24
Figure 26: Screenshot of search and filter by mood result	24
Figure 27: Screenshot of search and filter by tags	25
Figure 28: Screenshot of search and filter by tags result	25
Figure 29: Screenshot of showing streak related statistics on dashboard	26
Figure 30: Screenshot with light theme layout.....	27
Figure 31: Screenshot with dark theme layout	28
Figure 32: Screenshot of dashboard analytics displaying analytics visual	28
Figure 33: Screenshot of dashboard analytics displaying analytics visual	29
Figure 34: Screenshot of registration authentication	29
Figure 35: Screenshot of login authentication	30
Figure 36: Screenshot showing preview of export journal	31
Figure 37: Screenshot showing the export function.....	32
Figure 38: Screenshot showing the downloaded pdf	32
Figure 39: Registration wireframe	33
Figure 40: Login wireframe.....	33
Figure 41: Journal page wireframe	34
Figure 42: Calendar preview wireframe.....	35

Figure 43: Timeline view page wireframe	35
Figure 44: Search and filter page wireframe.....	36
Figure 45: Dashboard page wireframe	37
Figure 46: Settings page wireframe.....	38
Figure 47: Export page wireframe	38
Figure 48: Login UI.....	39
Figure 49: Registration UI.....	39
Figure 50: Journal page entry UI	40
Figure 51: Journal Entry page UI.....	40
Figure 52: Calendar preview	41
Figure 53: Paginated journal list page UI.....	41
Figure 54: Search and filter page UI.....	42
Figure 55: Dashboard page UI	42
Figure 56: Dashboard page UI	43
Figure 57: Export page UI	43
Figure 58: Settings page UI.....	44
Figure 59: Use case diagram	44
Figure 60: Entity relationship diagram	45
Figure 61: Milestone chart.....	46
Figure 62: Vcs repository.....	47
Figure 63: Commit history	47
Figure 64: Folders of commit.....	48
Figure 65: Code snippet	48
Figure 66: Screenshot showing clearly named files	49
Figure 67: Screenshot showing clear filed names	50
Figure 68: Screenshot showing clear file names	50
Figure 69: Screenshot showing descriptive naming conventions	51
Figure 70: Maintaining consistent formatting and indentation.....	52
Figure 71: Screenshot showing relevant and concise comments.....	53
Figure 72: Screenshot showing optimized database query	53
Figure 73: Screenshot showing avoidance of redundant inserts	53
Figure 74: Screenshots showing targeted data retrieval	53
Figure 75: Screenshot showing conditional rendering in UI	54
Figure 76: Screenshot showing conditional rendering in UI	54
Figure 77: Screenshot showing conditional rendering	54
Figure 78: Screenshot of clear separation of concerns	55
Figure 79: Screenshot of abstraction.....	55
Figure 80: Screenshot of dependency injection.....	55
Figure 81: Screenshot of business logic in service.....	56
Figure 82: Screenshot of service implementation in razor page.....	56
Figure 83: Screenshot of input validation error.....	57
Figure 84: Exception handling with try-catch block.....	57
Figure 85: Screenshot of error message	58
Figure 86: Screenshot of registration authentication error.....	58

Figure 87: Screenshot of login authentication error	59
Figure 88: Screenshot of null and boundary check code snippet	59
Figure 89: Test 1.1 (Screenshot of registration validation)	60
Figure 90: Test 1.2 (Screenshot of login validation):	61
Figure 91: Test 3(Screenshot of mood selection)	62
Figure 92: Test 3 (Screenshot of saved values at journal after mood selection)	62
Figure 93: Test 4 (Screenshot showing chosen tags)	63
Figure 94: Test 4 (Screenshot displaying assigned tags.)	64
Figure 95: Test 5(Screenshot writing formatted journal content)	65
Figure 96: Test 5 (Screenshot showing formatted journal content)	65
Figure 97: Test 6.1(Screenshot of journal creation)	66
Figure 98: Test 6.1(Screenshot showing the newly created entry with its date and timestamp)	67
Figure 99: Test 6.2 (Screenshot showing journal entry)	67
Figure 100: Test 6.3 (Screenshot showing content while updating)	68
Figure 101: Test 6.3 (Screenshot showing update option)	68
Figure 102: Test 6.3(Screenshot showing updated entry)	69
Figure 103: Test 6.4(Screenshot showing journal list after deletion)	69
Figure 104: Test 6.4 evidence	70
Figure 105: Test 7 (Screenshot showing the selected date and opened entry)	70
Figure 106: Test 8 (Screenshot of journal entries)	71
Figure 107: Test 8 (Screenshot of journal entries in a list with expanded content)	71
Figure 108: Test 9.1 (Screenshot showing search content)	72
Figure 109: Test 9.1 (Screenshot showing filtered search result)	72
Figure 110: Test 9.2 (Screenshot showing filtration content by date)	73
Figure 111: Test 9.2 (Screenshot showing filtered search result by date)	73
Figure 112: Test 9.3 (Screenshot of filtering by mood)	74
Figure 113: Test 9.3(Screenshot showing filtered search result by mood)	74
Figure 114: Test 9.2 (Screenshot of filtering by tag)	75
Figure 115: Test 9.4 (Screenshot showing filtered search result by tag)	76
Figure 116: Test 10 (Screenshot showing streak details on the dashboard)	77
Figure 117: Test 11 (Screenshot showing the preview pf journal before pdf export)	78
Figure 118: Test 11 (Screenshot showing the export process)	79
Figure 119: Test 11 (Screenshot showing the generated PDF)	79
Figure 120: Test 12 (Screenshot of current theme i.e light)	80
Figure 121: Test 12 (Screenshot of current theme i.e dark)	81

Table of tables

Table 1: Technology stack.....	4
Table 2: Features and functionalities.....	7
Table 3: Test 1.1	59
Table 4: Test 1.2	60
Table 5: Test 3	61
Table 6: Test 4	63
Table 7: Test 5	64
Table 8: Test 6.1	66
Table 9: Test 6.2	67
Table 10: Test 8	68
Table 11: Test 6.4	69
Table 12: Test 7	70
Table 13: Test 8	71
Table 14: Test 9.1	72
Table 15: Test 9.2	73
Table 17: Test 9.3	74
Table 19: Test 9.4	75
Table 20: Test 10	76
Table 21: Test 11	77
Table 22: Test 12	80

1 Introduction

It is well-exemplified that journaling is a good method to help one contemplate daily experiences, thoughts, and feelings. Technology is here to stay and so in a bigger proportion, most individuals have turned towards digital solutions, which are convenient, well organized and secure. The proposed project is related to the creation of a personal journaling desktop-based app in C#.NET that is to be convenient and systematic in terms of enabling users to record their everyday lives.

The application has more features than just a basic text entry as there are mood tracking, tagging and categorization. These features assist the user to structure his/her entries in a more efficient way and trace emotional or behavioral trends over time. The system will be set to have only one entry per day, motivating participants to consistently use their journals every day. It would be easier to monitor journaling and consistency.

The issue of user privacy is taken seriously all through the application. The personal journal data is confidential and the system has a secure system of access through a PIN or secure login mechanism. The entire data is kept locally with the help of SQLite database, which means that the users have full control over their data without having to rely on third-party services.

In short, the proposed project will provide a safe and convenient journaling system and will serve as the example of how the concepts of software development like application design, database integration, and user interface development may be put into practice in the real world.

1.1 Purpose

The application is developed as a secure and dependable digital space for personal journaling, allowing users to document their daily thoughts, feelings, and experiences in an organized manner. Beyond basic writing capabilities, the system incorporates features such as mood selection, tagging, and categorization to help users structure their entries and recognize patterns in their behavior over time. The application also offers visual overviews and analytical features, including mood

trends, journaling streak tracking, and tag usage summaries, which support meaningful reflection on personal progress. Throughout the design, user privacy and data security are given high priority, ensuring that all journal information is protected and remains entirely under the user's control.

1.2 Scope

- Users can create and manage only one journal entry per day.
- Each journal entry includes mood tracking, with one required primary mood and up to two optional additional moods.
- Tags and categories can be added to help organize and manage journal entries efficiently.
- Previous entries can be browsed using a calendar view as well as a list or timeline view.
- A dashboard displays summary insights such as mood trends, tag usage, and journaling activity.
- User data is protected through secure login or PIN-based access.
- The system is designed to support local data storage using SQLite in future development stages.

1.3 Objectives

- Users can create, view, edit, and delete their daily journal entries.
- The system supports rich text or Markdown formatting to enhance the writing experience.
- Each entry records one primary mood and up to two secondary moods, categorized as Positive, Neutral, or Negative.
- Journal entries can be organized using both built-in and user-defined tags.
- Past entries are accessible through a calendar view or a paginated list for easy navigation.
- Users can search and filter entries based on date, mood, tags, or text content.
- The application tracks daily journaling streaks and clearly marks missed days to encourage regular use.

- Analytics are provided, showing mood patterns, frequently used tags, and word count trends over time.
- Journal data is secured using password or PIN-based protection.
- Users have the option to export their journal entries in PDF format.
- An optional light and dark mode is available for a more personalized user interface.

1.4 Technology Stack

Layer	Technology / Tool	Purpose
Framework	MAUI Blazor Hybrid	Create and develop a cross-platform user interface using Blazor components to ensure consistent functionality and appearance across different devices and platforms.
UI / Styling	HTML and CSS	Plan and build the application's layout and visual style with a focus on consistency, clarity, and ease of use for users.
Version Control	Git	Manage and track modifications to the project's code throughout the development process to ensure updates are properly monitored and maintained.

Development IDE	Visual Studio 2022	Establish and manage the main development environment used to write, execute, and test the application's code.
Future Integration	SQLite	Store journal entries locally on the user's device so they remain available for use in future phases of the project.
Libraries	Blazor, .NET MAUI	Develop and provide reusable UI components and icon libraries to ensure consistent use across the entire application.

Table 1: Technology stack

2 Features and functionalities

2.1 Feature Breakdown

Feature	Description	Implementation / UI Guidance	Completion Status
Journal Entry Management	Enable users to create, update, or delete one journal entry per day, with timestamps automatically saved for when entries are created or modified.	A straightforward CRUD-style interface that includes options like Add Entry, Edit Entry, and Delete Entry, with the system automatically generating the date and time.	Completed
Rich Text / Markdown Editor	Supports styled text input with features such as headings, bullet lists, bold and italic text, and clickable links.	Built using a C# RichTextBox or Markdown editor, complete with a toolbar that provides text formatting options.	Completed
Mood Tracking	Users must choose one primary mood, with the option to add up to two secondary moods if desired.	Mood selection is handled through dropdown lists or radio buttons, with moods organized into Positive, Neutral, and Negative groups.	Completed
Tag Management	Allows journal entries to be	Tags can be chosen using a multi-select	Completed

	organized using a mix of predefined tags and tags created by the user.	dropdown or a tag input field, allowing users to add their own custom tags as needed.	
Calendar-Based Navigation	Lets users access and open previous journal entries by choosing a specific date.	A calendar feature on the main interface that loads the corresponding journal entry when a date is selected.	Completed
Paginated Journal Listing	Shows journal entries in an organized list or timeline view to simplify browsing and navigation.	Implemented with a ListView or DataGridView, including pagination controls like Next and Previous for easier navigation.	Completed
Search and Filtering	Allows users to search entries by text and filter them based on date, mood, or tags.	Features a search box and filter controls that automatically refresh the displayed entries based on user input.	Completed
Streak Monitoring	Monitors consecutive journaling days, records the longest streak, and highlights days	Streak details are shown on the dashboard and automatically updated according to	Completed

	when no entries were made.	the user's journaling activity.	
Theme Customization	Provides the option to switch between light, dark, or personalized custom themes.	Built with a theme switcher that consistently applies the selected color scheme throughout the application.	Completed
Analytics Dashboard	Offers analytics including mood patterns, frequently used tags, and changes in word count over time.	Visualized using libraries like LiveCharts.	Completed
Security and Privacy	Secures journal entries with password or PIN-based authentication.	Features a login screen that uses securely hashed credentials to block unauthorized access.	Completed
Journal Exporting	Allows users to export selected journal entries as PDF files for a specified date range.	Includes a PDF export feature that lets users select a date range and generate files using different libraries.	Completed

Table 2: Features and functionalities

2.2 Description of features with evidence

2.2.1 CRUD (Create,Read,Update,Delete) for Journal Entries

Description: This feature lets users completely manage their daily journal entries. They can create a new entry for the day, update it later if needed, view previous entries, or delete an entry when it's no longer required. Every action is recorded with automatic timestamps for creation and last modification, giving users a clear record of their journaling history.

Evidence:

- Journal Entry Page Render

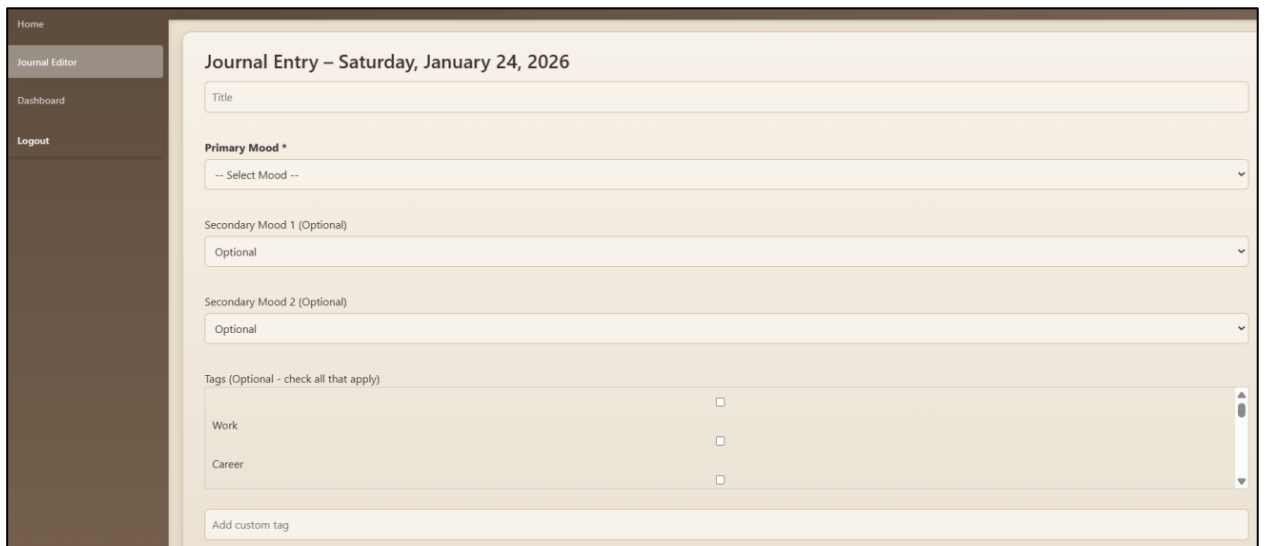
The screenshot displays a web application interface for journaling. On the left is a dark brown sidebar with white text links: 'Home', 'Journal Editor' (highlighted), 'Dashboard', and 'Logout'. The main content area has a light beige background. At the top, it says 'Journal Entry – Saturday, January 24, 2026'. Below this is a form with several sections: a 'Title' text input; a 'Primary Mood *' dropdown menu with '-- Select Mood --'; a 'Secondary Mood 1 (Optional)' dropdown menu with 'Optional'; a 'Secondary Mood 2 (Optional)' dropdown menu with 'Optional'; and a 'Tags (Optional - check all that apply)' section with checkboxes for 'Work' and 'Career', and a text input for 'Add custom tag'. A vertical scrollbar is visible on the right side of the tags section.

Figure 1: Journal page entry load

After logging in successfully, the app opens the Journal page, where users can create new journal entries.

- Journal entry CRUD (Save)

Journal Entry – Saturday, January 24, 2026

Note: You've already created an entry for today. You can edit it below.

A happy day!

Primary Mood *

Happy (Positive)

Secondary Mood 1 (Optional)

Excited (Positive)

Secondary Mood 2 (Optional)

Relaxed (Positive)

Tags (Optional - check all that apply)

Work

Career

Add custom tag

Journal Content (Markdown)

I am very happy today.

Figure 2: Journal Creation

Once a journal entry is successfully created and saved, a preview appears with the note “journal created for today.” From here, users can proceed to edit the entry if needed.

- Screenshot of saved journal

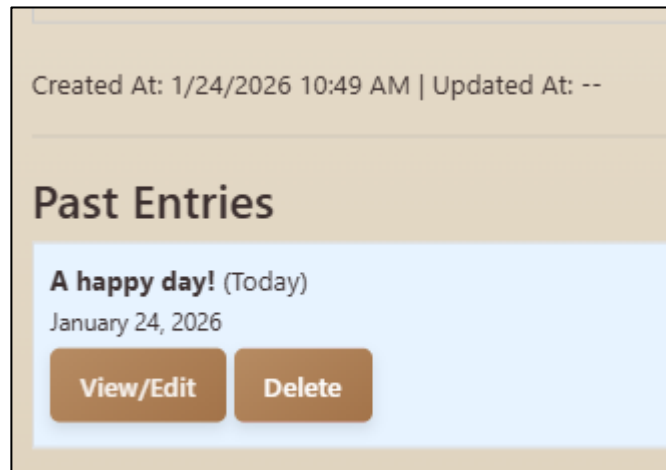


Figure 3: Saved journal preview

After successfully saving a newly created journal, users can view the entry along with its content. Additionally, options to edit, view, or delete the journal are available alongside the saved entry.

- Screenshot of journal entry (View)

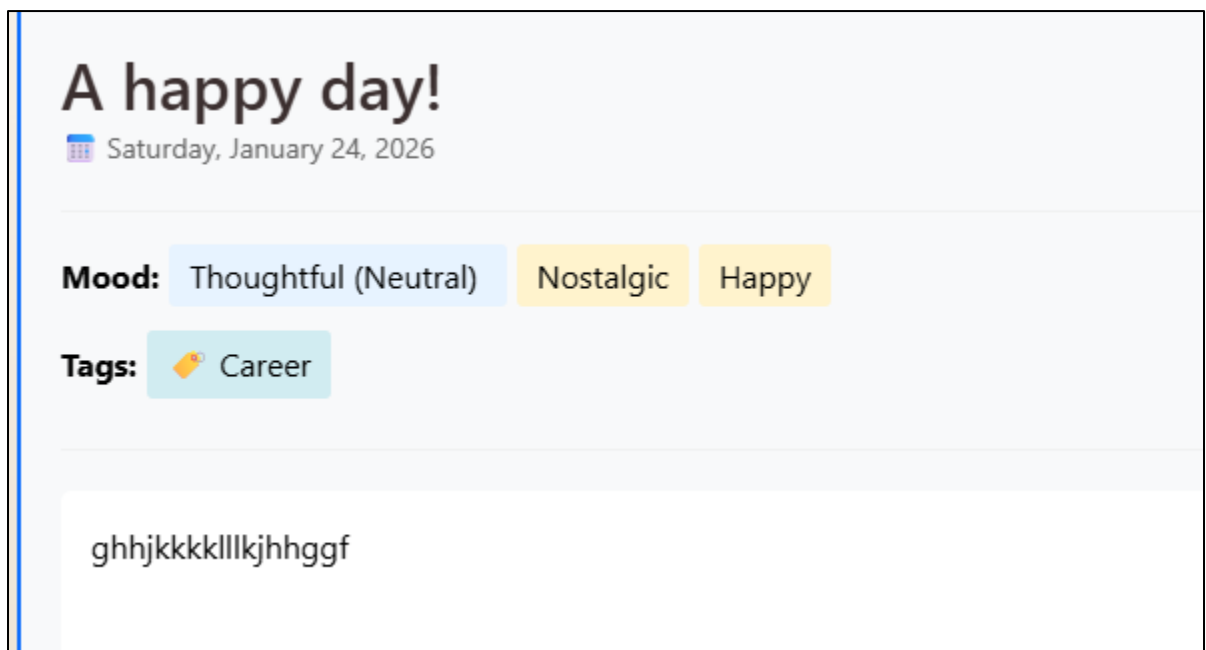
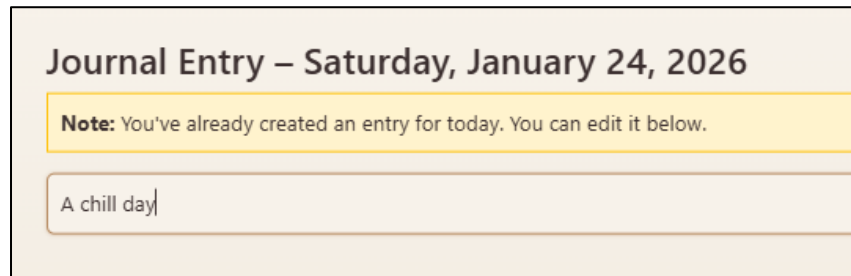


Figure 4: View journal entry

- Screenshot of journal entry (Update)



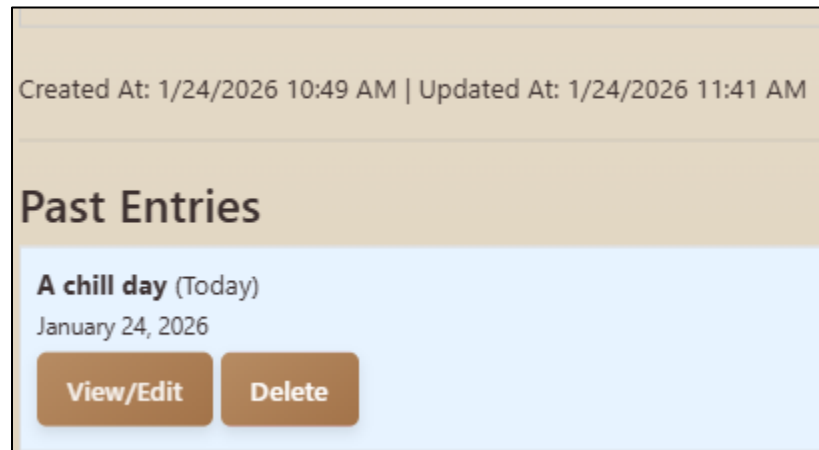
Journal Entry – Saturday, January 24, 2026

Note: You've already created an entry for today. You can edit it below.

A chill day

Figure 5: Journal entry update

- Screenshot of update shown



Created At: 1/24/2026 10:49 AM | Updated At: 1/24/2026 11:41 AM

Past Entries

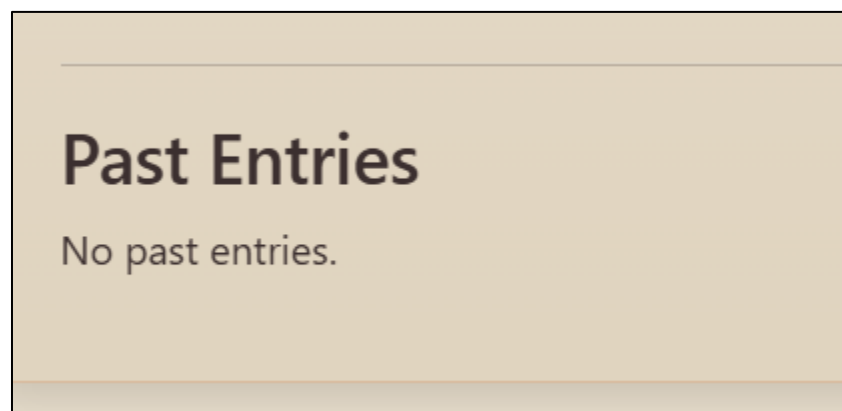
A chill day (Today)
January 24, 2026

[View/Edit](#) [Delete](#)

Figure 6: Journal update preview

The journal title was updated, and the preview then displayed the entry along with the updated date.

- Journal entry (Delete)



Past Entries

No past entries.

Figure 7: Journal deletion

2.2.2 Rich Text/ Markdown writing support

Description: To improve usability and writing flexibility, the editor supports both rich-text and Markdown formatting. Users can format their content using options such as bold, italics, headings, lists, and hyperlinks. This enables users to present their thoughts in a clearer and more organized manner.

Evidence:

*Figure 8: screenshot of journal entry markdown feature*

The journal was written in formatted text and later it showed in preview.



Figure 9: Screenshot of a journal entry displaying formatted text.

2.2.3 Mood Tracking

Description: This feature lets users record their emotional state for each journal entry. Users are required to choose one main mood that reflects their primary feeling, and they can optionally select up to two additional moods for more context. Moods are divided into Positive, Neutral, and Negative categories, allowing users to review and analyze trends over time. Tracking moods helps users understand their emotional patterns and supports reflection on overall mental well-being.

Evidences:

- Journal Page Mood Drop down
- Primary Mood

Primary Mood *
-- Select Mood --
-- Select Mood --
Happy (Positive)
Excited (Positive)
Relaxed (Positive)
Grateful (Positive)
Confident (Positive)
Calm (Neutral)
Thoughtful (Neutral)
Curious (Neutral)
Nostalgic (Neutral)
Bored (Neutral)
Sad (Negative)
Angry (Negative)
Stressed (Negative)
Lonely (Negative)
Anxious (Negative)

Figure 10: Mood dropdown content

- Secondary Mood

Secondary Mood 1 (Optional)
Optional
Optional
Happy (Positive)
Excited (Positive)
Relaxed (Positive)
Grateful (Positive)
Confident (Positive)
Calm (Neutral)
Thoughtful (Neutral)
Curious (Neutral)
Nostalgic (Neutral)
Bored (Neutral)
Sad (Negative)
Angry (Negative)
Stressed (Negative)
Lonely (Negative)
Anxious (Negative)

Figure 11: Secondary mood dropdown

This is a mood dropdown that allows users to select from various moods organized into categories. It includes options for both a primary mood and secondary moods.

- **Selection of mood**

Primary Mood *
Relaxed (Positive) ▼
Secondary Mood 1 (Optional)
Calm (Neutral) ▼
Secondary Mood 2 (Optional)
Thoughtful (Neutral) ▼

Figure 12: Mood selection

Mood is selected through the dropdown to fill journal.

Journal Entry – Monday, January 26, 2026

A good day!
Monday, January 26, 2026

Mood: Relaxed (Positive) Calm Thoughtful

Tags: Self-care Finance Weather

So how was today?

- Good

1. Very good

Hi there
Hey

Created: 1/26/2026 8:47 PM | Last Updated: Never

Figure 13: Mood selected shown in journal

Then, the selected mood is shown in journal after creating it.

2.2.4 Login / Registration

Description: This feature guarantees that the journal application remains secure and private. Users can create an account by entering a username and password, which are safely stored using encryption. During login, credentials are verified before access is granted, preventing unauthorized users from viewing personal journal entries. This feature forms the core of the application's security and safeguards sensitive user information.

Evidence:

- Registration page load

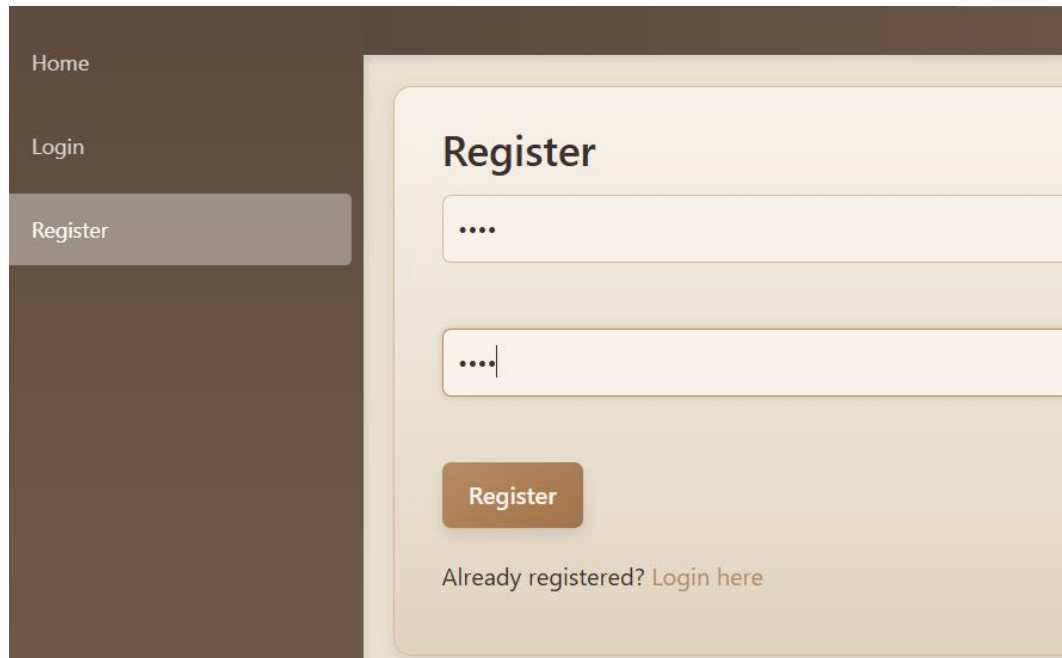


Figure 14: Registration page load

The app opens the registration page, where users can enter and save the PINs they want to remember.

- Login page render

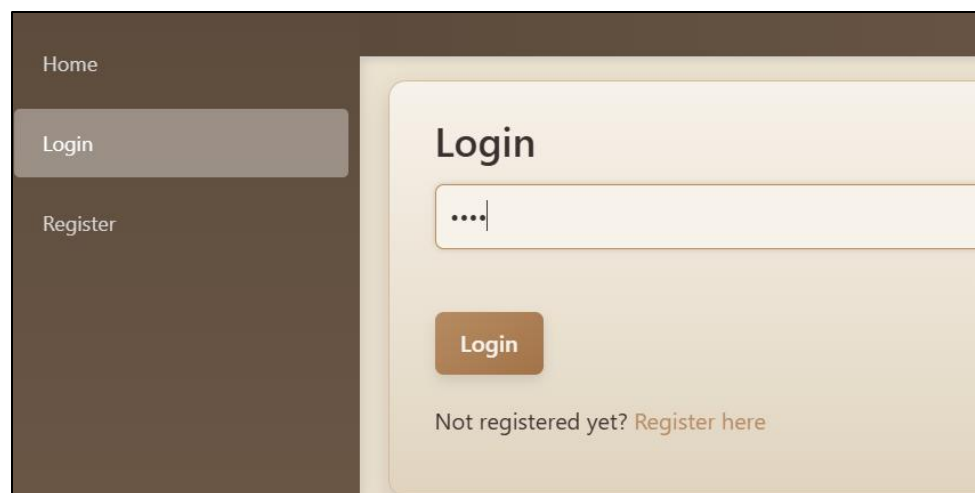


Figure 15: Login page render

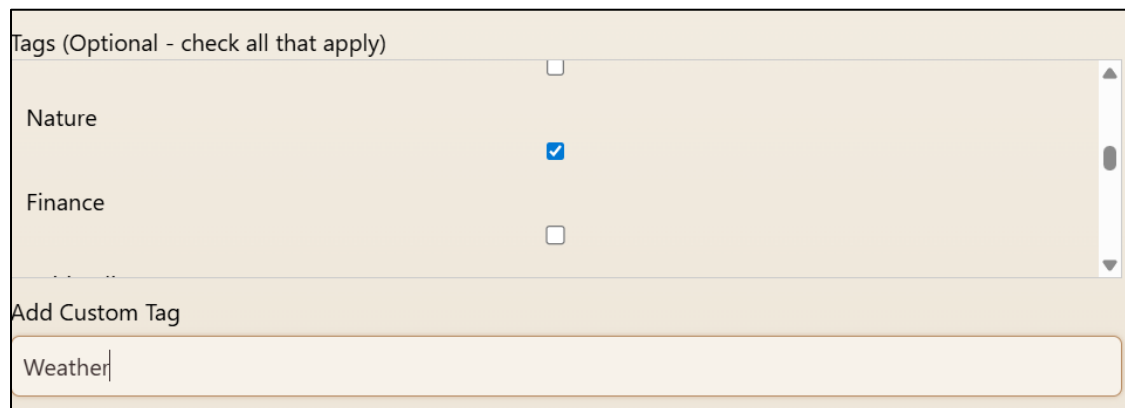
After completing registration, the app navigates to the login page, where users can enter their PIN to access the dashboard.

2.2.5 Tagging system

Description: The tagging feature allows users to organize their journal entries more effectively. Users can choose from predefined tags like Work, Health, Travel, and Fitness for quick categorization, or create custom tags to suit their individual needs. Tags simplify searching and filtering of entries and also provide useful information for analytics, helping users see which aspects of their life they write about most often.

Evidence:

- Tag Selection with custom tag



The screenshot shows a user interface for selecting tags. At the top, it says "Tags (Optional - check all that apply)". Below this is a list of predefined tags: "Nature", "Finance", and "Fitness". Each tag has a checkbox next to it. The "Nature" checkbox is checked, while "Finance" and "Fitness" are unchecked. Below the list of predefined tags is a section labeled "Add Custom Tag". Inside this section is a text input field containing the word "Weather".

Figure 16: Tag selection with custom tag

Users can select a required tag and also enter a custom tag of their choice.

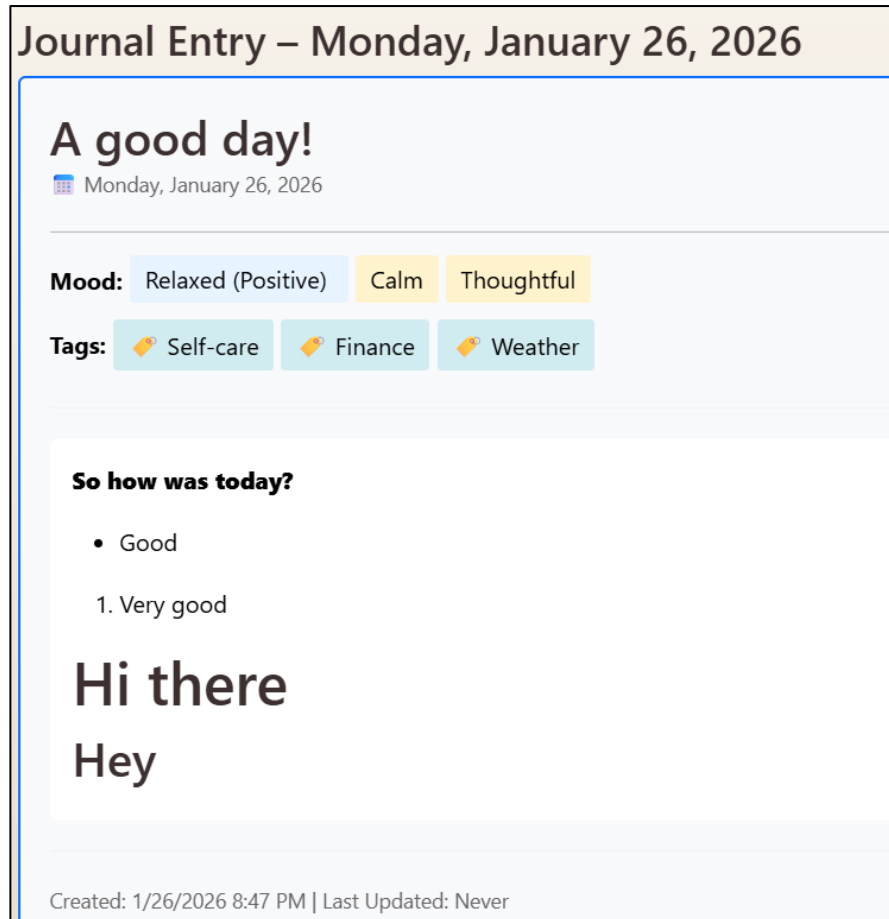


Figure 17: Selected tag shown in journal

2.2.6 Calendar Preview

Description: The Calendar Navigation feature lets users browse and access past journal entries through a calendar interface. Each calendar date corresponds to a day in the journal, with dates that have entries visually highlighted so users can easily see which days contain content. By selecting a specific date, users can view, edit, or delete the associated journal entry. This feature enhances the application's usability by offering a fast and intuitive way to find entries, particularly for users who keep long-term journaling records.

Evidence:

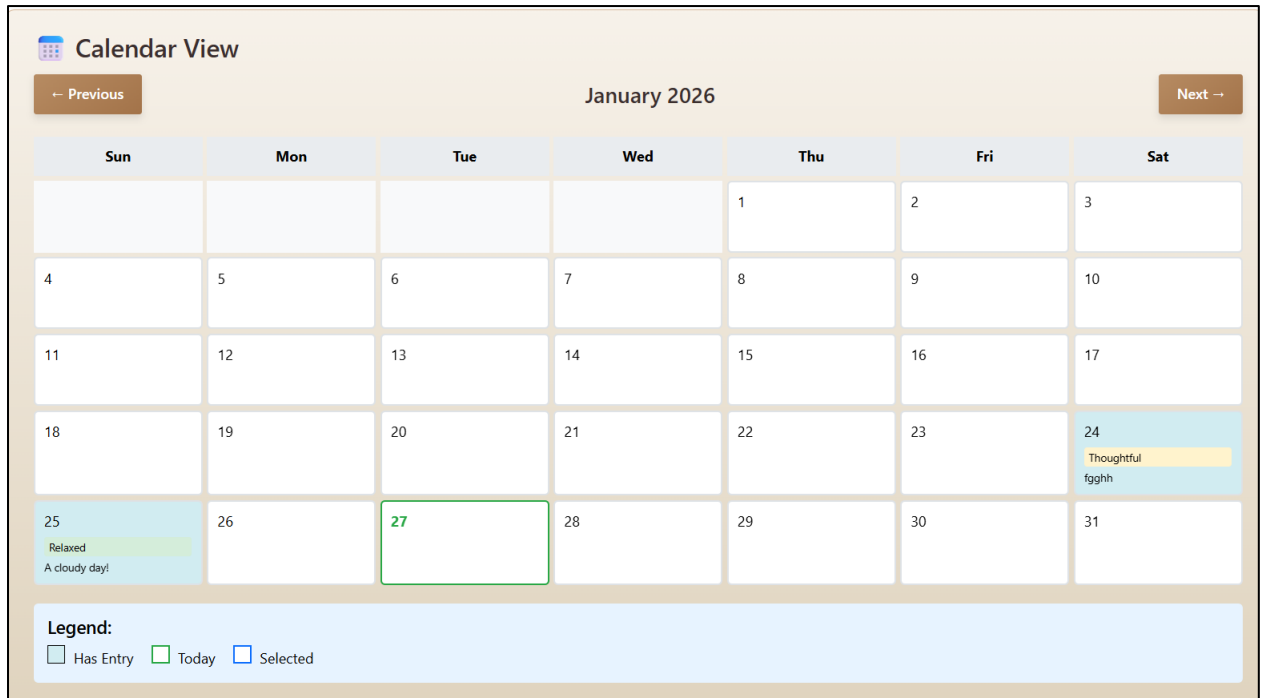


Figure 18: Calendar preview

The highlighted dates on the calendar indicate the days on which journal entries have been made, displaying each entry's title on its respective day. Users can navigate to a specific journal entry by selecting the corresponding date.

2.2.7 Paginated Journal View

Description: Besides the calendar view, the application provides a paginated list or timeline view that shows journal entries in chronological order. Users can navigate between pages using pagination controls, making it easy to browse entries efficiently as the number of entries increases.

Evidence:

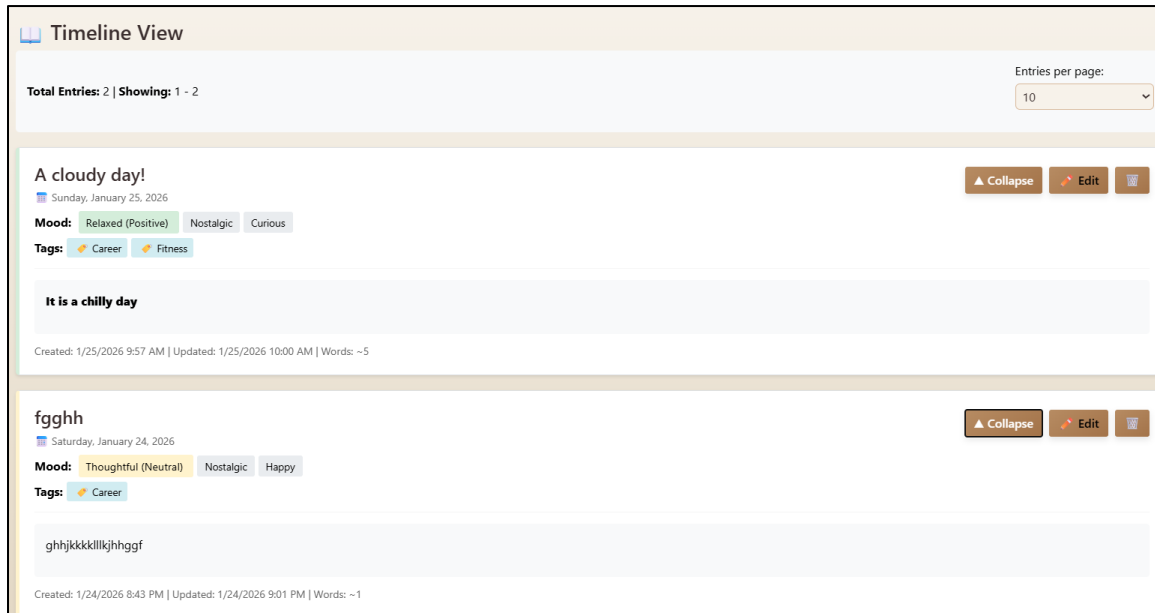


Figure 19: Screenshot showing the paginated journal list with navigation controls.

The above screenshot is the evidence of paginated journal list of journal entries created till date.

2.2.8 Search and filter

Description: Users can search for journal entries using keywords found in the title or content. The system also provides filtering options based on date range, selected moods, and tags. These tools help users quickly locate specific entries without manually browsing through the entire journal history.

Evidence: Screenshot displaying the search bar and applied filter options.

Search & Filter

Search

Search by title or content...

▼ Advanced Filters

Date Range

mm/dd/yyyy

to

mm/dd/yyyy

Filter by Mood

All Moods

Mood Type

All Types

Filter by Tags

All Tags

☒

Work

☐

Career

☐

Apply Filters

Clear All

Results: 2 of 2 entries

A cloudy day!

Sunday, January 25, 2026

Mood:

Relaxed (Positive)

Nostalgic

Curious

Tags:

Career

Fitness

****It is a chilly day****

fghhh

Saturday, January 24, 2026

Mood:

Thoughtful (Neutral)

Nostalgic

Happy

Tags:

Career

Figure 20: Screenshot of search and filter page

Search & Filter

Search

Figure 21: Screenshot displaying the search bar and applied filter options.



Figure 22: Screenshot displaying the result after searching

If we search the related content in our search bar, we get to see the result of entry we are trying to search downside of the search and filter page.

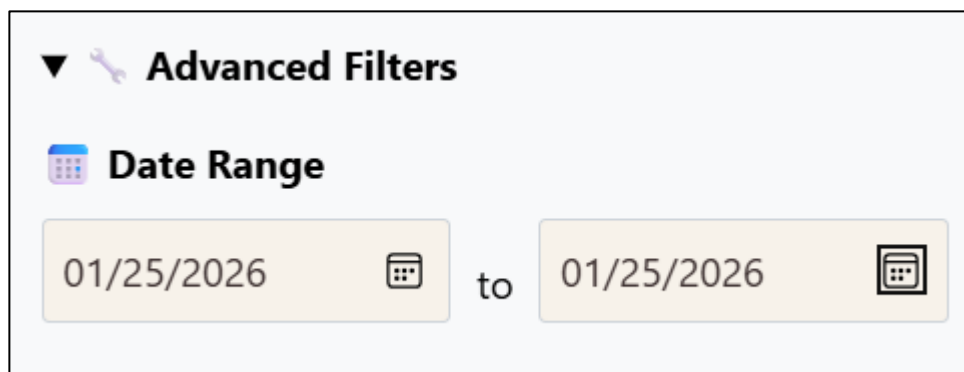
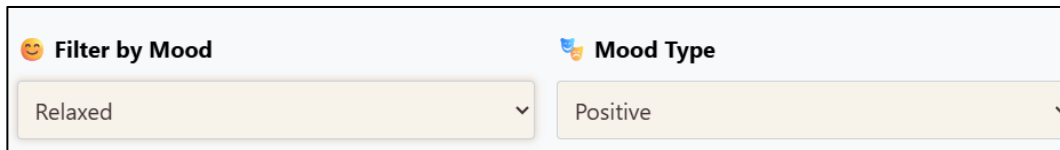


Figure 23: Screenshot of searching and filtering by date range



Figure 24: Screenshot of search and filter result by date range

The above screenshot shows the narrowing of entry selected by date range.



The screenshot shows a filter interface with two sections: "Filter by Mood" and "Mood Type". The "Filter by Mood" section has a dropdown menu with "Relaxed" selected. The "Mood Type" section has a dropdown menu with "Positive" selected.

Figure 25: Screenshot of search and filter by mood



The screenshot shows the results of a search and filter by mood. At the top, it says "Results: 1 of 2 entries (Filters active)". Below this, there is a section titled "A cloudy day!" with a date "Sunday, January 25, 2026". Under "Mood:", there are three buttons: "Relaxed (Positive)" (highlighted in green), "Nostalgic", and "Curious". Under "Tags:", there are two buttons: "Career" and "Fitness". At the bottom, there is a section titled "It is a chilly day".

Figure 26: Screenshot of search and filter by mood result

The above screenshot showed the result of filtration of entry by moods we entered while writing our journal.

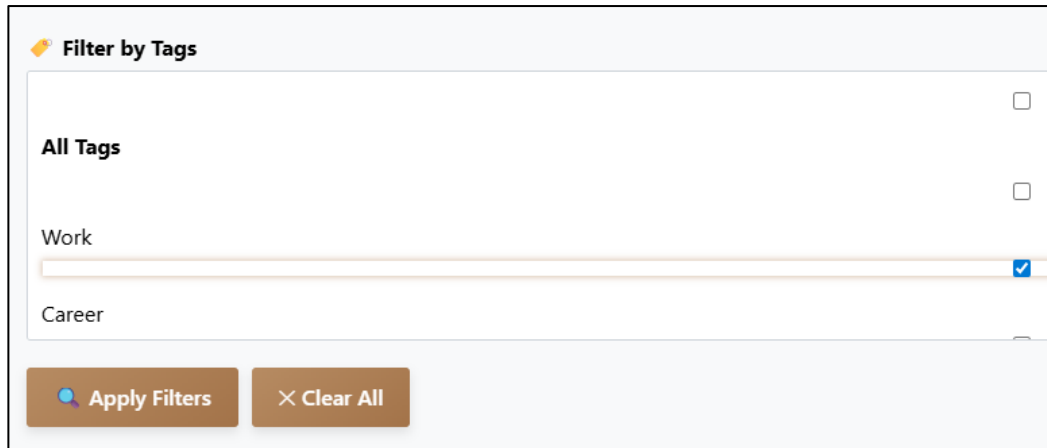


Figure 27: Screenshot of search and filter by tags

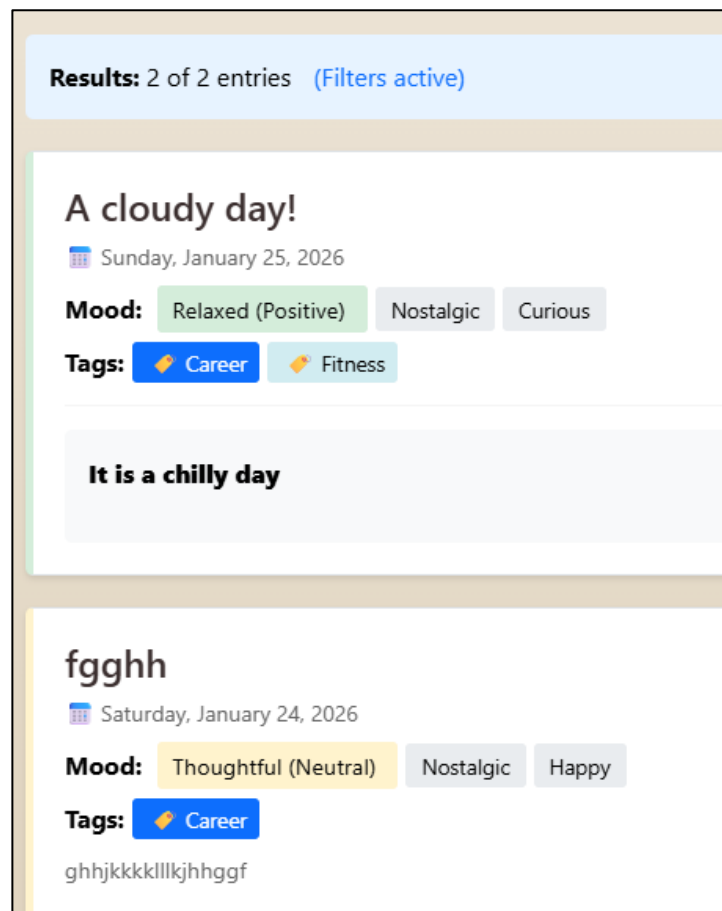


Figure 28: Screenshot of search and filter by tags result

It shows the result of journal entry by searching and filtering of tag.

2.2.9 Streak Tracking

Description: To encourage regular journaling, the application automatically monitors the user's current journaling streak, the longest streak achieved, and any missed days. These values update dynamically based on daily usage, providing clear feedback on journaling consistency.

Evidence: Screenshot showing streak-related statistics on the dashboard.

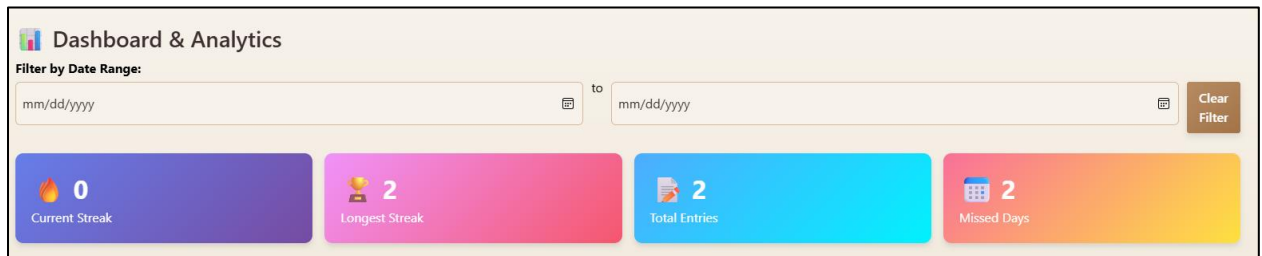


Figure 29: Screenshot of showing streak related statistics on dashboard

2.2.10 Theme Customization

Description: The application includes theme options that allow users to switch between light and dark modes. This feature improves visual comfort, especially during extended use, and gives users the ability to personalize the appearance of the application according to their preference.

Evidence: Screenshot comparing the light and dark theme layouts.

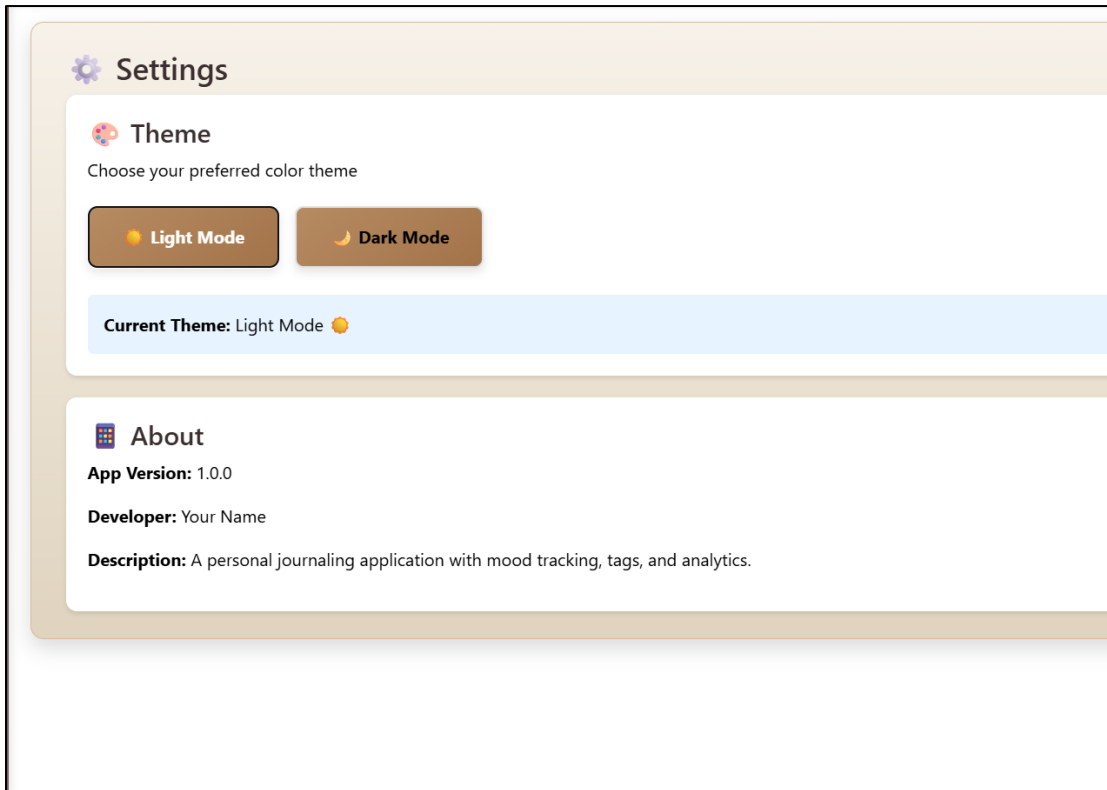


Figure 30: Screenshot with light theme layout

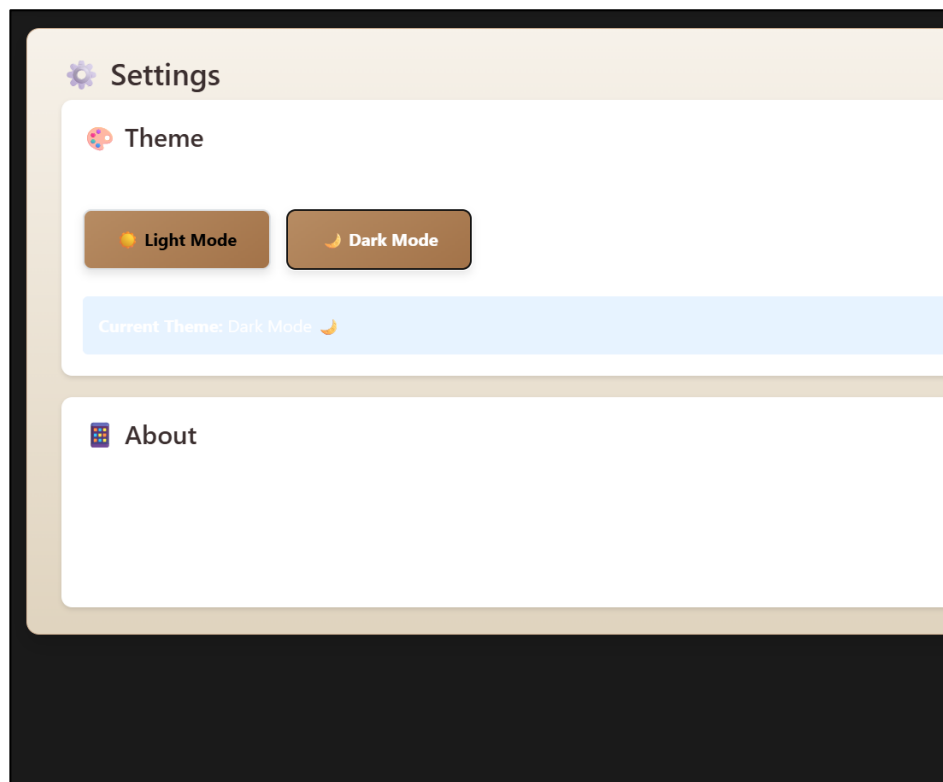
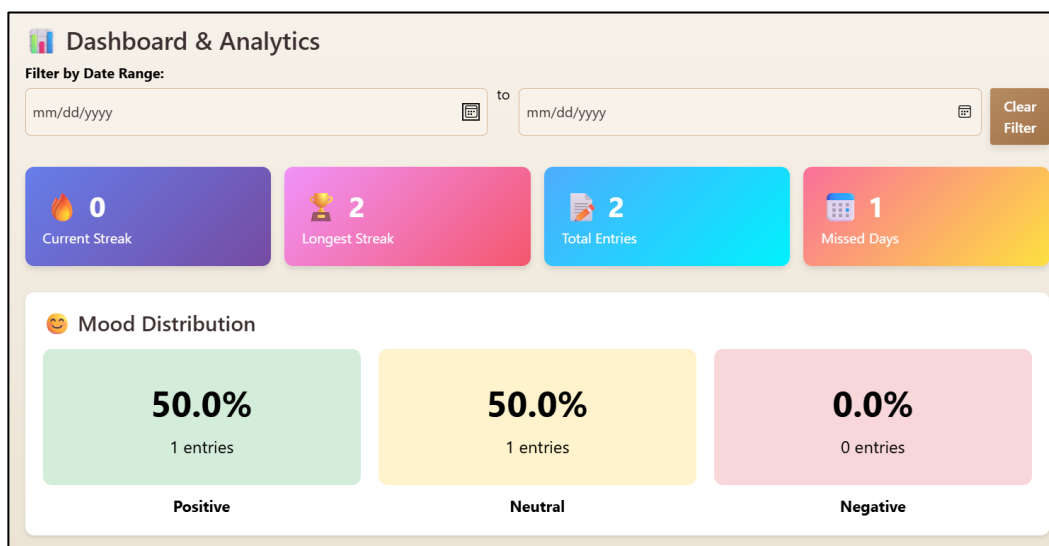


Figure 31: Screenshot with dark theme layout

2.2.11 Dashboard Analytics and Insights

Description: A dedicated dashboard presents visual summaries and analytical insights such as mood distribution, most frequently recorded moods, commonly used tags, tag breakdown by category, and word count trends over time. Users can apply date range filters to focus on specific periods, making the analytics more relevant and informative.

Evidence: Screenshot of the dashboard displaying charts and analytical visuals.

*Figure 32: Screenshot of dashboard analytics displaying analytics visual*

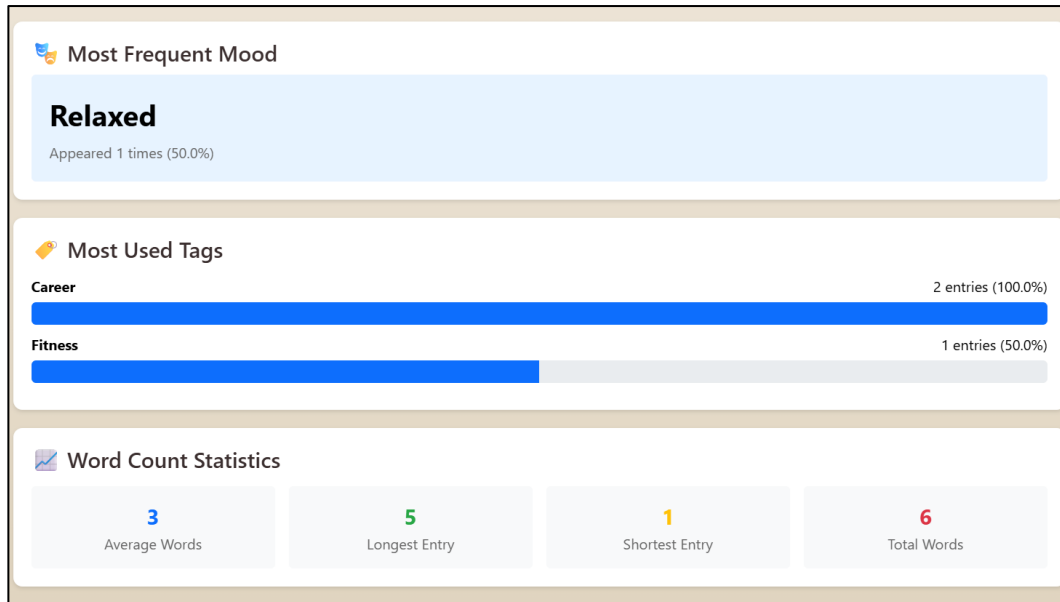


Figure 33: Screenshot of dashboard analytics displaying analytics visual

2.2.12 Security and Privacy

Description: User privacy is protected through password or PIN-based authentication. Access to journal content is restricted until valid credentials are entered, ensuring that personal data remains confidential and secure from unauthorized access.

Evidence:

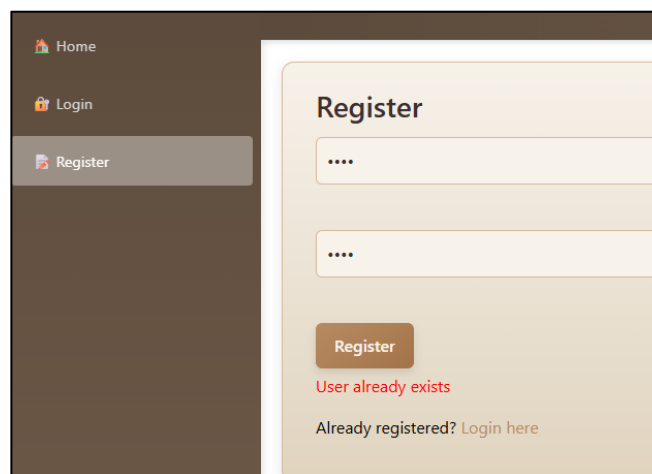


Figure 34: Screenshot of registration authentication

Registration validation thrown after trying to register again with the registered pin.

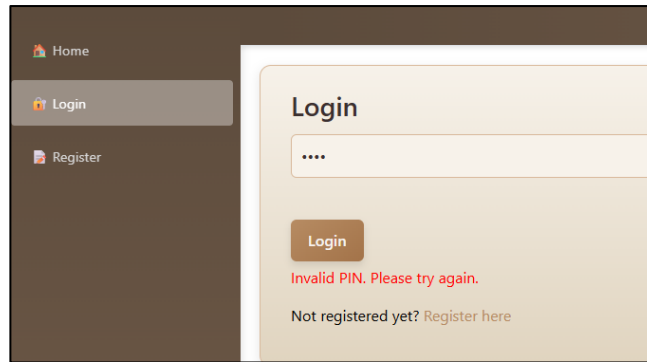


Figure 35: Screenshot of login authentication

Login validation thrown after password mismatch.

2.2.13 Exporting Journal Entries as PDF

Description: The application allows users to export their journal entries as PDF files by selecting a desired date range. This feature supports offline access, long-term storage, and personal backups, giving users greater control over their journal data.

Evidence: Screenshot showing the export option and a generated PDF file.

Select Date Range

Start Date:

01/24/2026

End Date:

01/24/2026

 Preview

 Export & Share

Preview: 1 entries will be exported (Jan 24, 2026 - Jan 24, 2026)

Preview

fgggh

 Saturday, January 24, 2026

Mood: Thoughtful (Neutral)

ghhjkkkklllkjhggf

Figure 36: Screenshot showing preview of export journal

Here, the preview of journal entry to be exported is shown after selecting date range.

Then the share and export option is shown before downloading.

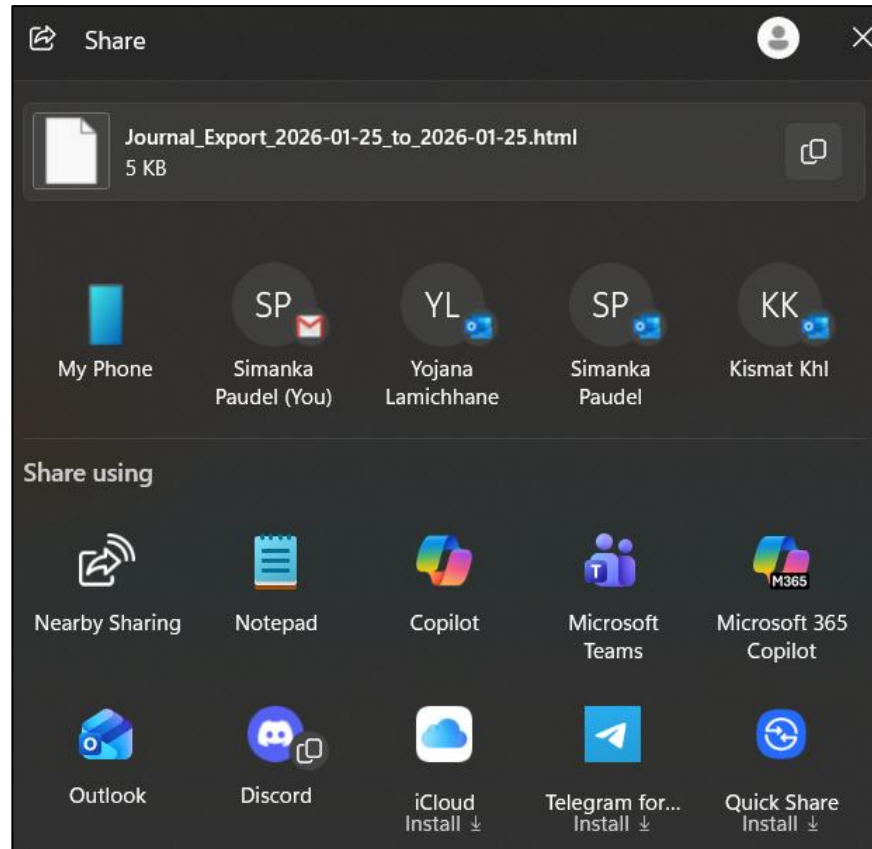


Figure 37: Screenshot showing the export function

After successfully downloading the pdf, it is presented like this in pdf format.



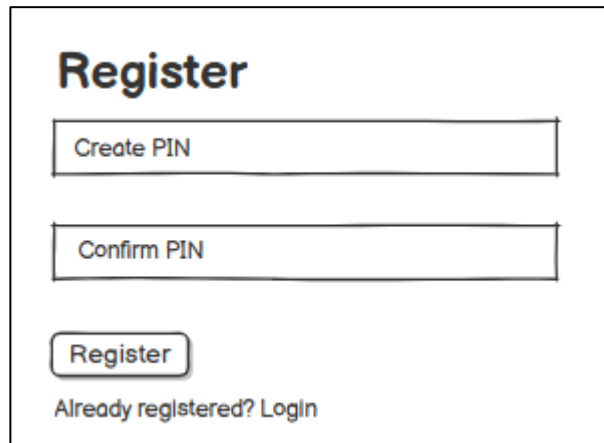
Figure 38: Screenshot showing the downloaded pdf

3 Proof of work

3.1 Design

3.1.1 Wireframes

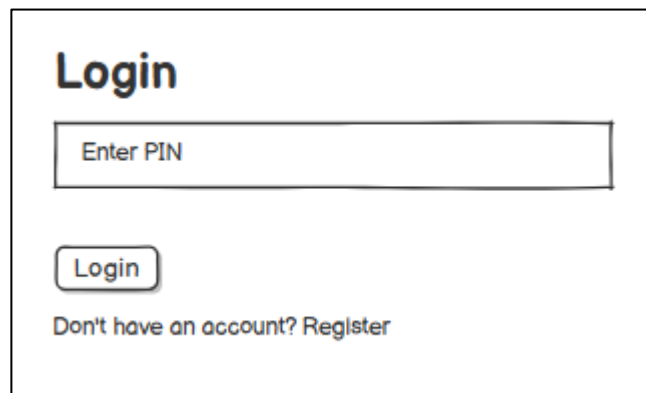
3.1.1.1 Page: Register



A wireframe for a registration page. It features a title "Register" at the top. Below the title are two text input fields: the first is labeled "Create PIN" and the second is labeled "Confirm PIN". Below these fields is a rounded rectangular button labeled "Register". At the bottom of the form, there is a link that says "Already registered? Login".

Figure 39: Registration wireframe

3.1.1.2 Page: Login/Pin



A wireframe for a login page. It features a title "Login" at the top. Below the title is a text input field labeled "Enter PIN". Below this field is a rounded rectangular button labeled "Login". At the bottom of the form, there is a link that says "Don't have an account? Register".

Figure 40: Login wireframe

3.1.1.3 Page: Journal Editor

Journal Entry - Date

Title

Mood Tracking
[dropdown](#) : Primary Mood
Secondary Mood

Tags
☐ Work ☐ Health ☐ Travel ☐ Fitness
Custom Tag:

B I U 1 H1 H2 Link

Write the journal entry here....

Word Count: ____ words

< Save Entry < Update Entry < Delete Entry

Created At: ____ Updated At: ____

Figure 41: Journal page wireframe

3.1.1.4 Page: Calendar View

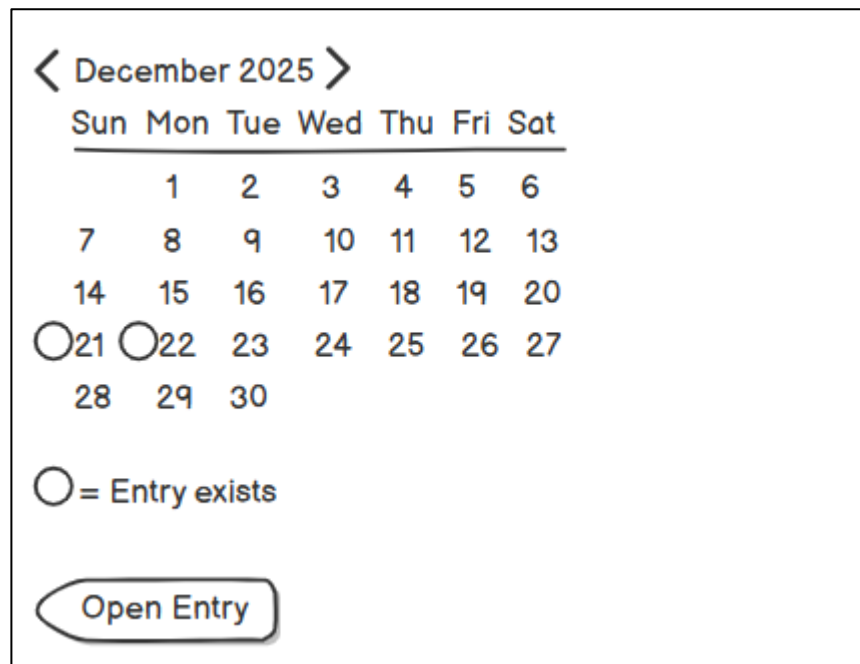


Figure 42: Calendar preview wireframe

3.1.1.5 Paginated journal list page wireframe

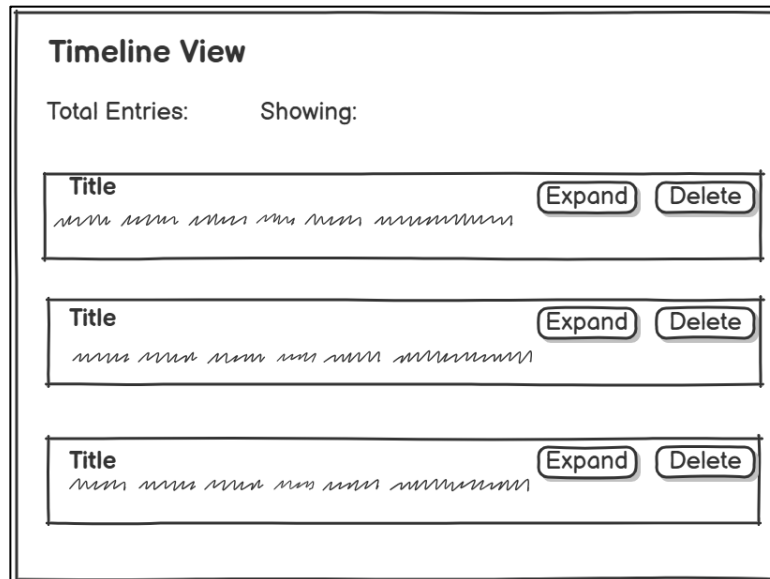


Figure 43: Timeline view page wireframe

3.1.1.6 Search and filter page wireframe

Search & Filter

Search

Date Range

mm/yy/dd to mm/yy/dd

Filter by moods

All moods ▾

Mood type

All types ▾

Filter by Tags

All tags

Apply Filters

Results

Figure 44: Search and filter page wireframe

3.1.1.7 Dashboard Page wireframe

Dashboard & Analytics

Filter by date range:

to

Clear Files

2
Current
Streak

2
Largest
Streak

2
Total
Entries

1
Missed
Days

Mood distribution

25 %
Positive

50%
Neutral

25%
Negative

Most frequent mood

Mood A

Most used tags

Tag A

5

Word Count statistics

3 Average words	5 Longest Entry	1 Shortest entry	6 Total Words
-----------------------	-----------------------	------------------------	---------------------

Figure 45: Dashboard page wireframe

3.1.1.8 Settings Page Wireframe

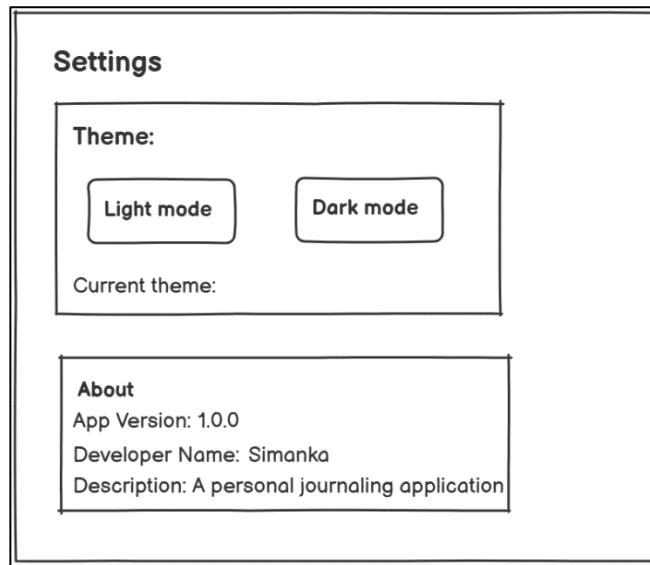


Figure 46: Settings page wireframe

3.1.1.9 Export page wireframe



Figure 47: Export page wireframe

3.1.2 UI Designs

3.1.2.1 Login UI

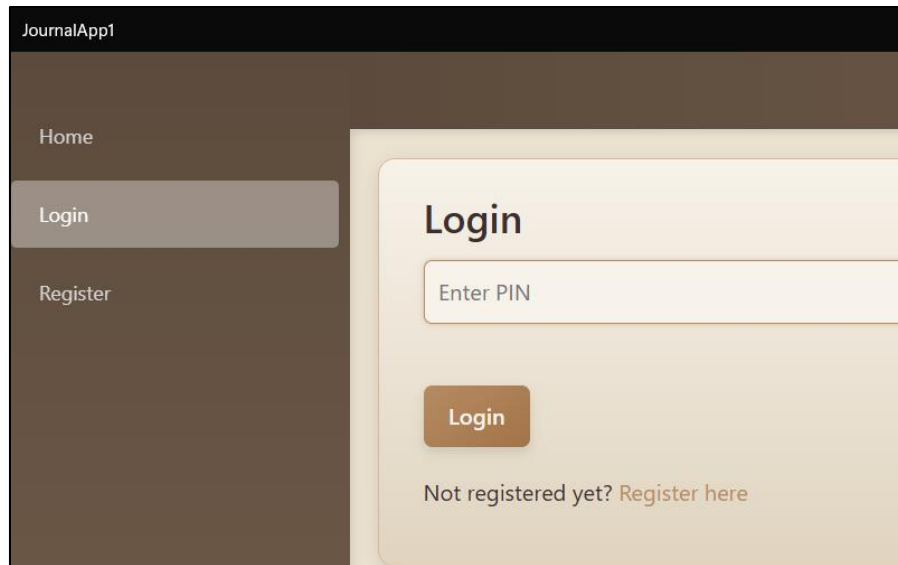


Figure 48: Login UI

3.1.2.2 Register UI

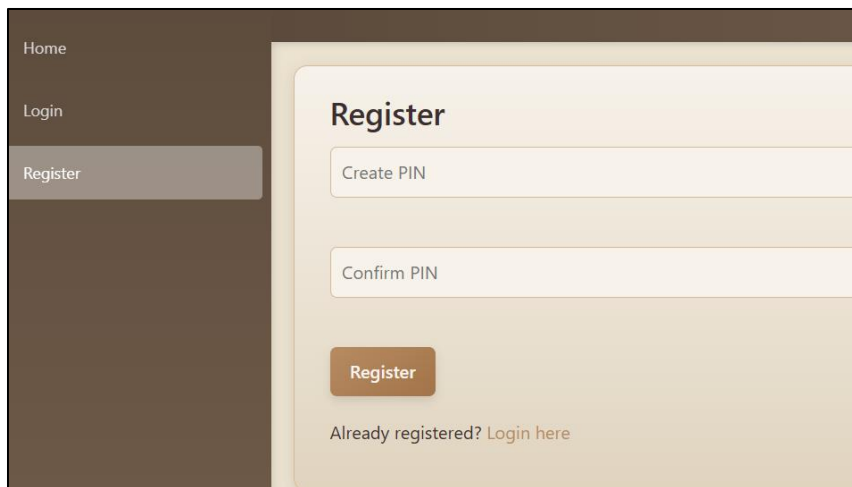


Figure 49: Registration UI

3.1.2.3 Journal entry page UI

Home

Journal Editor

Dashboard

Logout

Journal Entry – Saturday, January 24, 2026

Title

Primary Mood *

-- Select Mood --

Secondary Mood 1 (Optional)

Optional

Secondary Mood 2 (Optional)

Optional

Tags (Optional - check all that apply)

Work ☐

Career ☐

Add custom tag

Figure 50: Journal page entry UI

Journal Content (Markdown)

Use Markdown: **bold**, *italic*, # Heading

Save Entry

Preview

Created At: 1/24/2026 10:26 AM | Updated At: --

Past Entries

No past entries.

Figure 51: Journal Entry page UI

3.1.2.4 Calendar preview page UI

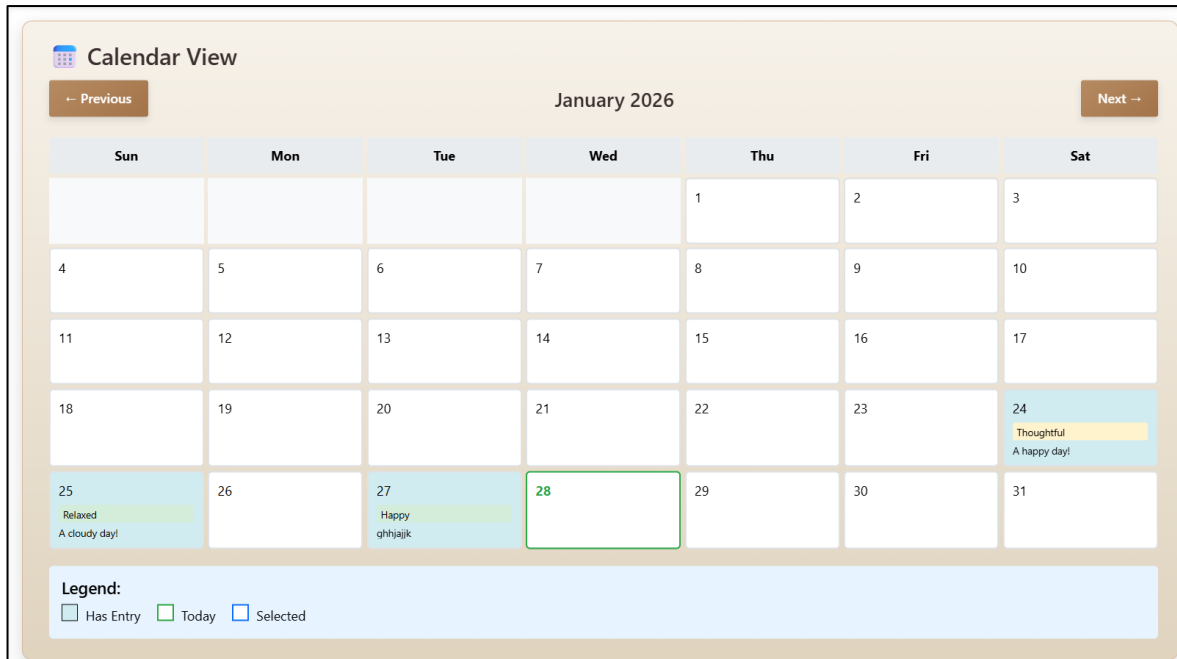


Figure 52: Calendar preview

3.1.2.5 Paginated journal list page UI

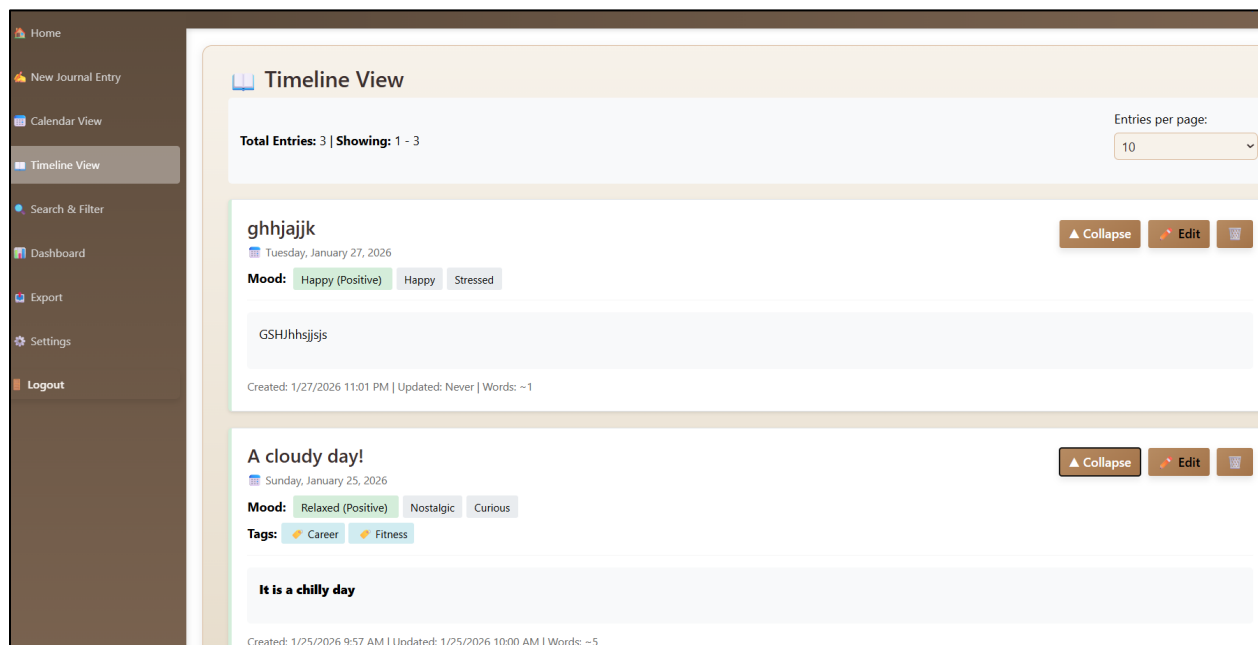


Figure 53: Paginated journal list page UI

3.1.2.6 Search and filter page UI

Search & Filter

Search by title or content...

Advanced Filters

Date Range: mm/dd/yyyy to mm/dd/yyyy

Filter by Mood: All Moods

Mood Type: All Types

Filter by Tags:

- Exercise ☐
- Reading ☐
- Writing ☐

Apply Filters Clear All

Figure 54: Search and filter page UI

3.1.2.7 Dashboard page UI

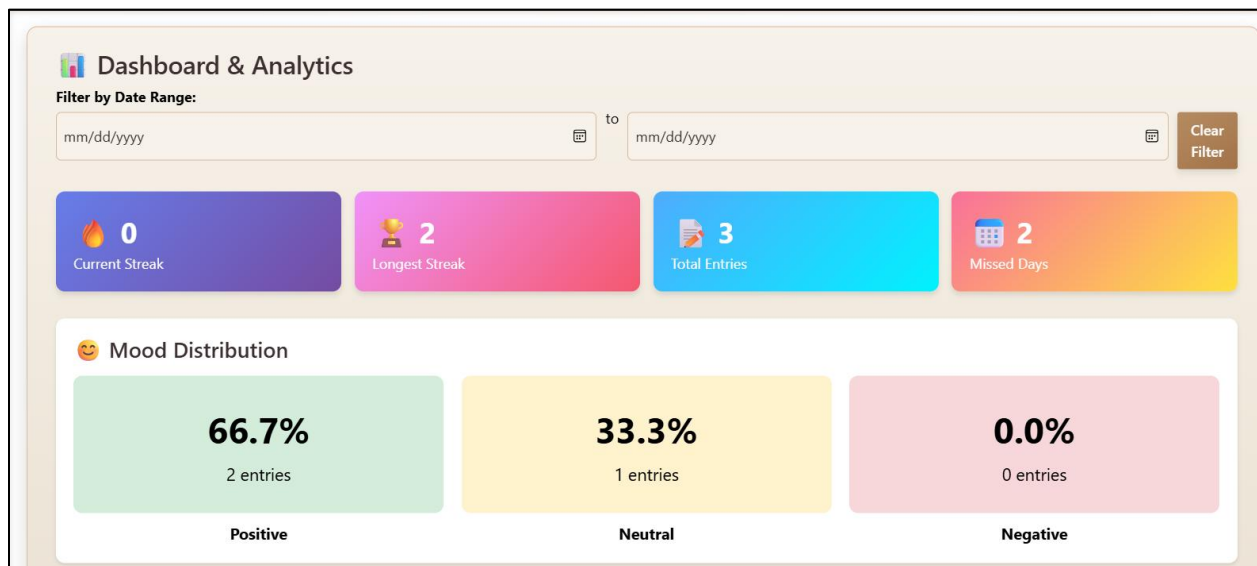
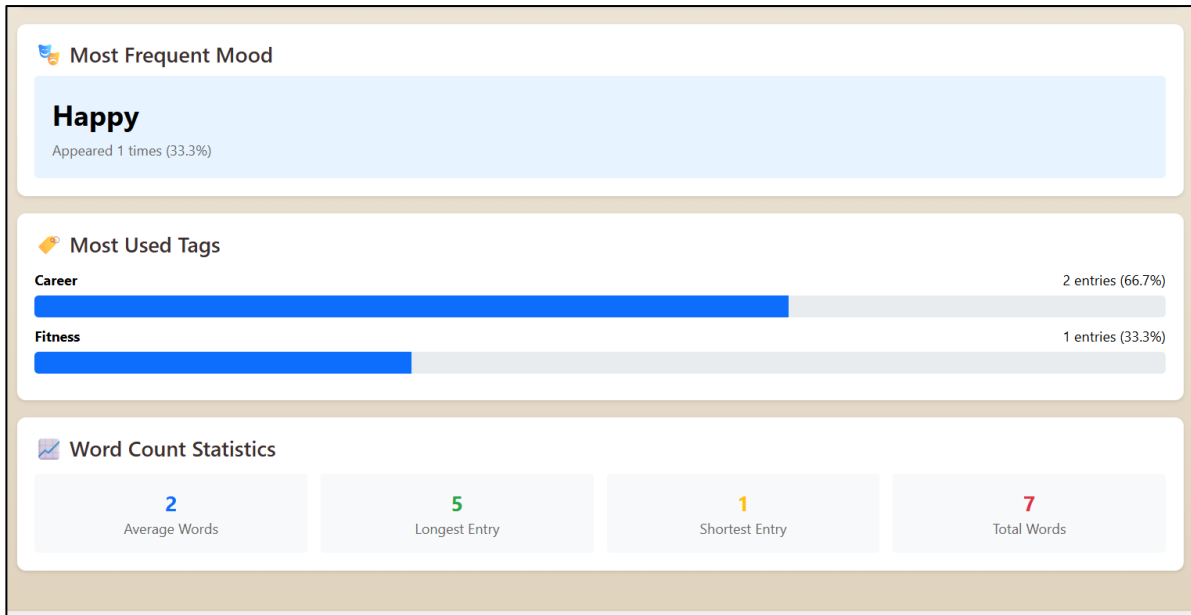
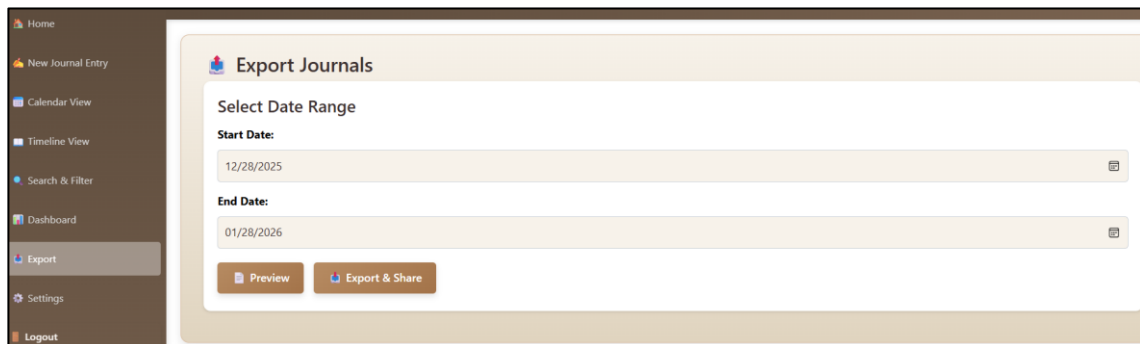


Figure 55: Dashboard page UI

*Figure 56: Dashboard page UI*

3.1.2.8 Export page UI

*Figure 57: Export page UI*

3.1.2.9 Settings page UI

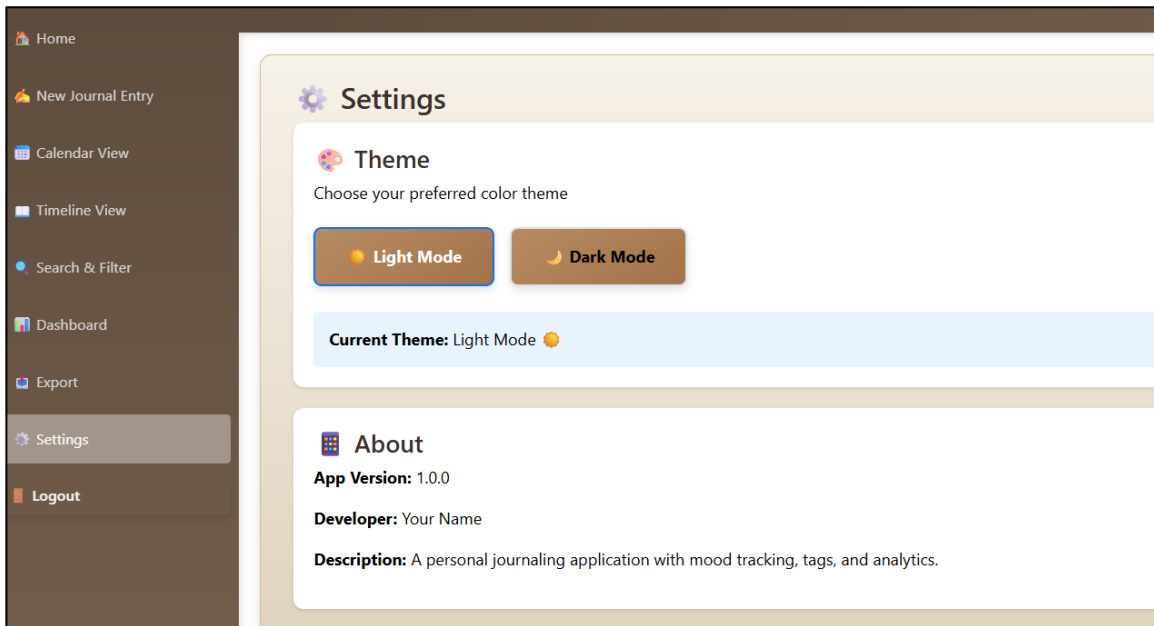


Figure 58: Settings page UI

3.1.3 Use Case Diagram

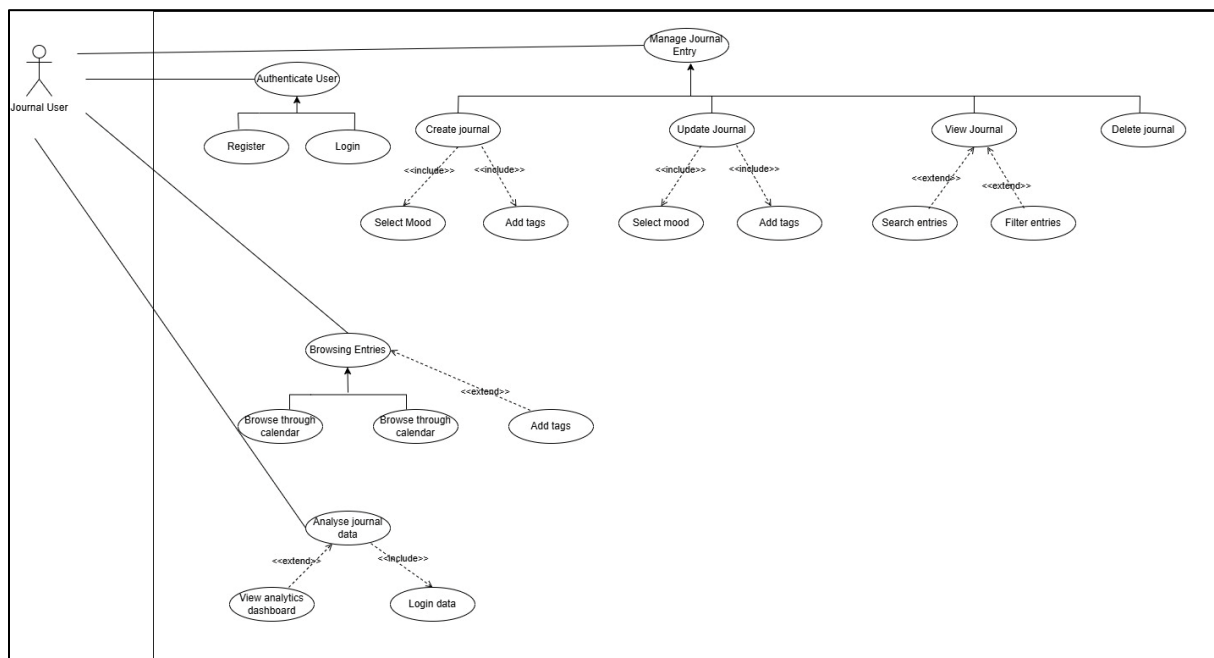


Figure 59: Use case diagram

3.1.4 Entity Relationship Diagram

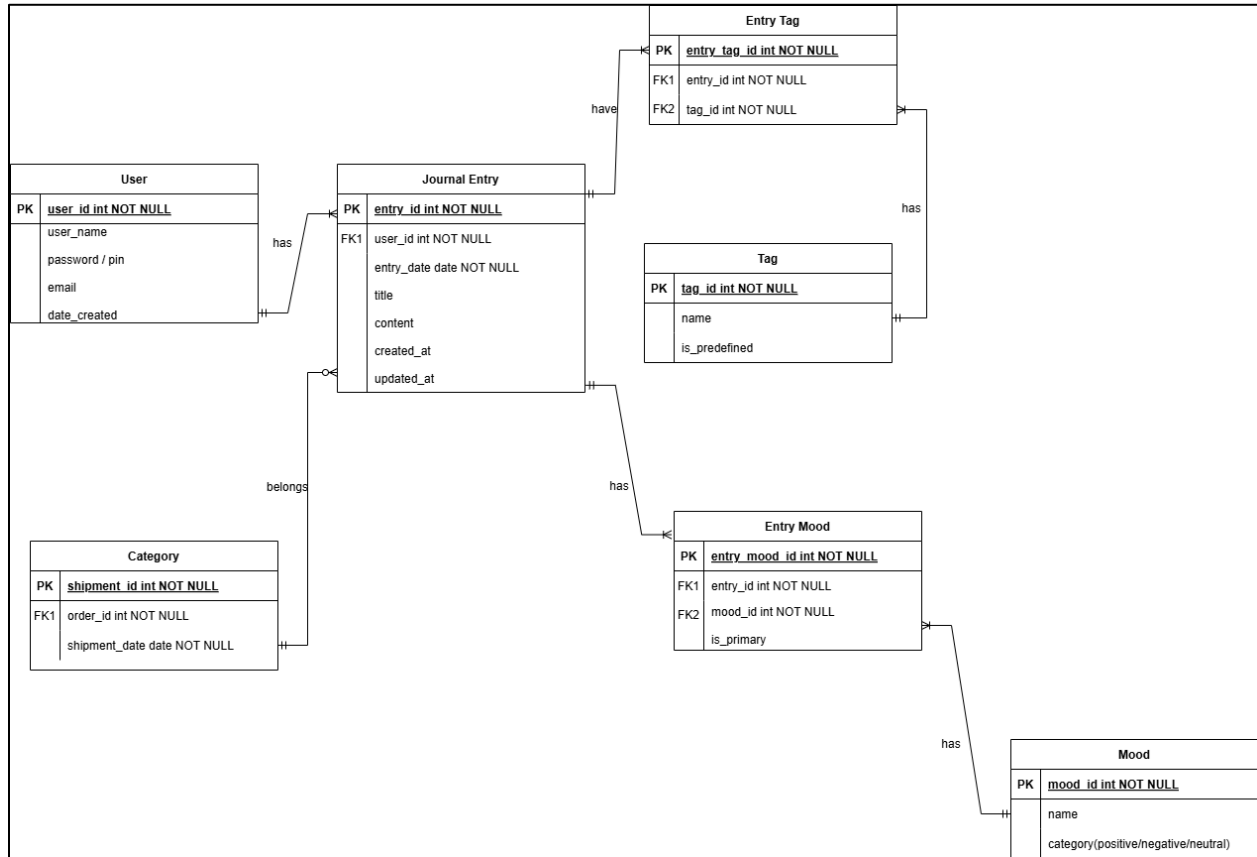


Figure 60: Entity relationship diagram

3.1.5 Milestone chart

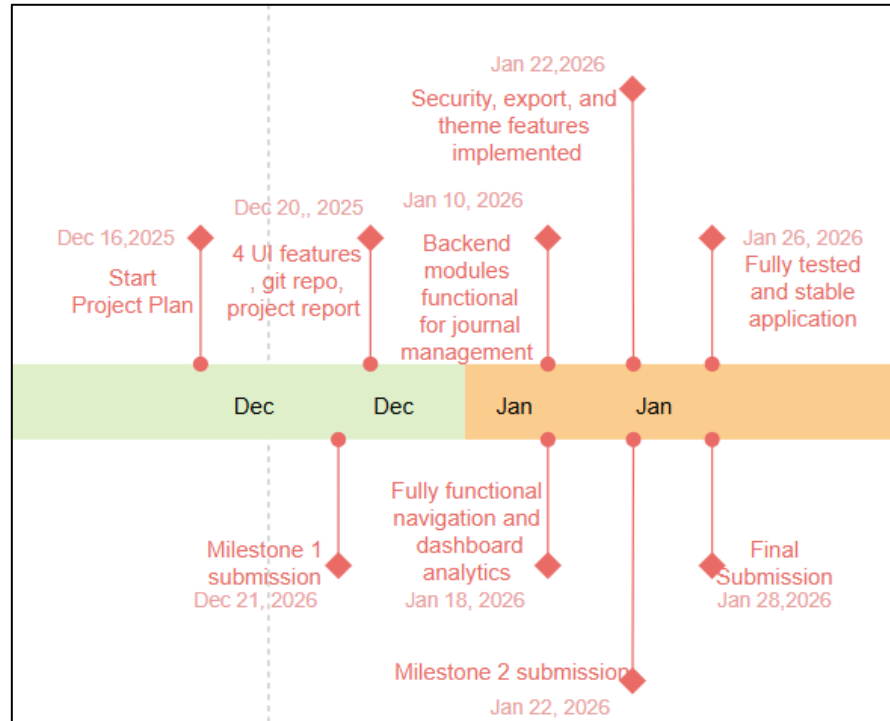


Figure 61: Milestone chart

3.2 Vcs repository

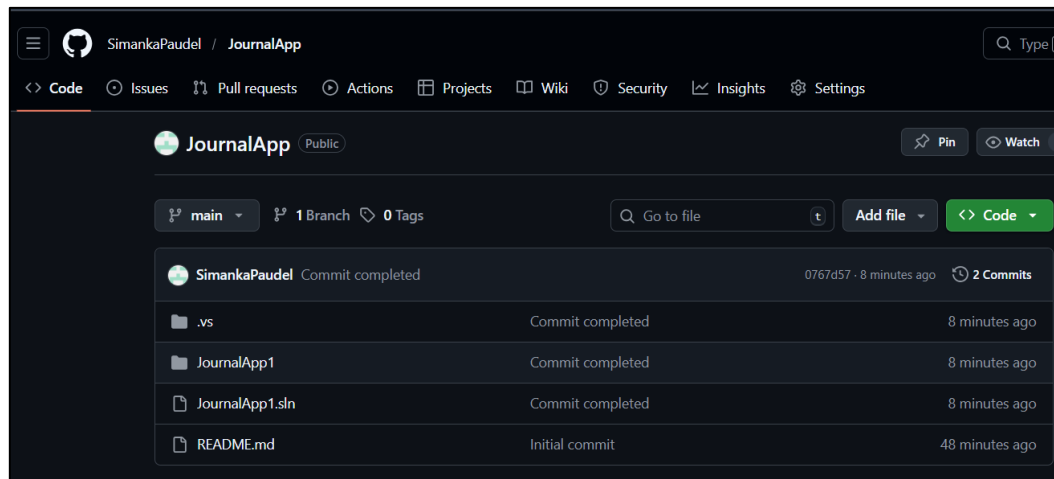


Figure 62: Vcs repository

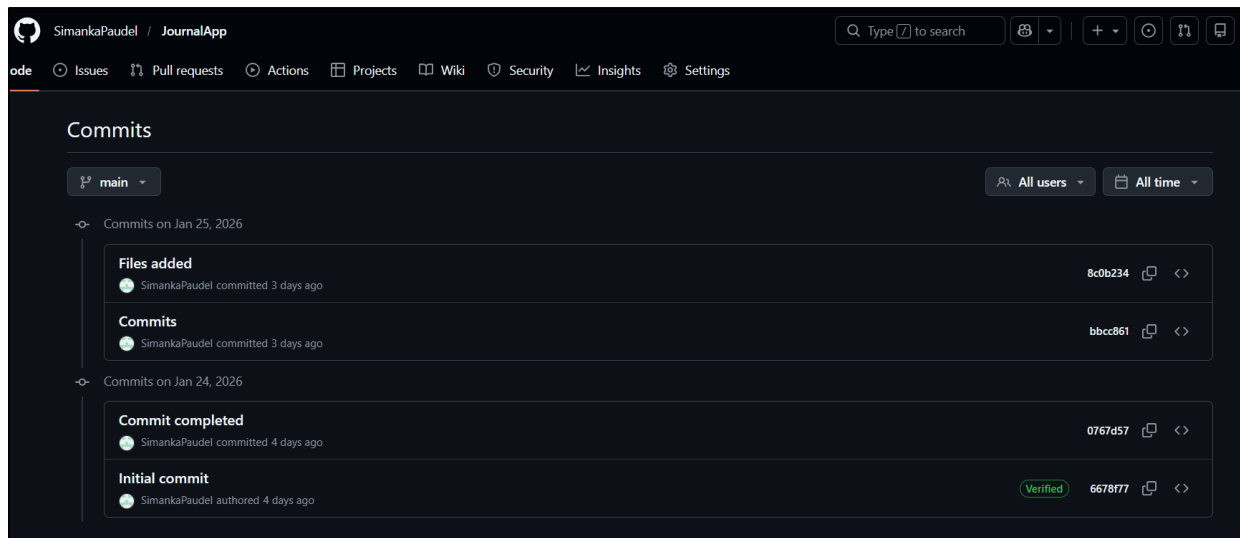


Figure 63: Commit history

JournalApp / JournalApp1 /			↑ Top
Name	Last commit message	Last commit date	
..			
Components	Files added	3 days ago	
Data	Commits	3 days ago	
Helpers	Files added	3 days ago	
Models	Commits	3 days ago	
Platforms	Commit completed	4 days ago	
Properties	Commit completed	4 days ago	
Resources	Commit completed	4 days ago	
Services	Files added	3 days ago	
bin/Debug/net9.0-windows10.0.19041.0/win10-x64	Files added	3 days ago	
obj	Files added	3 days ago	
wwwroot	Commit completed	4 days ago	
App.xaml	Commit completed	4 days ago	
App.xaml.cs	Commit completed	4 days ago	

Figure 64: Folders of commit

```

Code Blame 142 lines (123 loc) · 5.6 KB
1  using SQLite;
2  using System.IO;
3  using System.Threading.Tasks;
4  using System.Collections.Generic;
5  using System.Linq;
6  using JournalApp1.Models;
7  using System;
8
9  namespace JournalApp1.Data
10 {
11     public class AppDatabase
12     {
13         private static SQLiteAsyncConnection _database;
14         private static readonly SemaphoreSlim _initlock = new SemaphoreSlim(1, 1);
15         private static bool _isInitialized = false;
16
17         public SQLiteAsyncConnection Database => _database;
18
19         // Ensure database is initialized before use
20         public async Task EnsureInitializedAsync()
21         {
22             if (_isInitialized)
23                 return;
24
25             await _initlock.WaitAsync();
26             try
27             {
28                 if (_isInitialized)
29                     return;
30
31                 var dbPath = Path.Combine(FileSystem.AppDataDirectory, "journal.db");
32                 _database = new SQLiteAsyncConnection(dbPath);

```

Figure 65: Code snippet

3.3 Quality

3.3.1 Code Readability

Care was taken to ensure the code is easy to read and understand. Clear and descriptive names were used for files, classes, methods, and variables so their purpose is immediately obvious. Consistent spacing and indentation were maintained across all files, and comments were added only where they help explain important or complex logic. This makes the code easier to follow and update in the future.

Evidence:

- Well organized source files

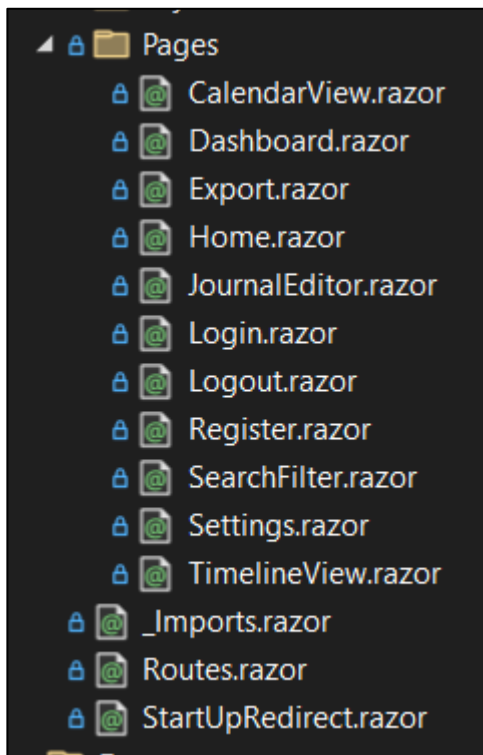


Figure 66: Screenshot showing clearly named files

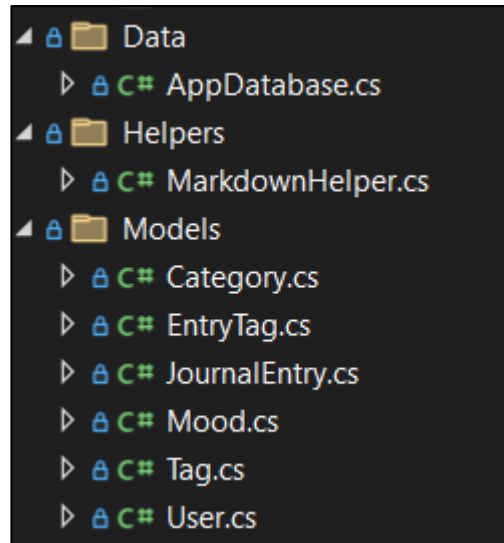


Figure 67: Screenshot showing clear filed names

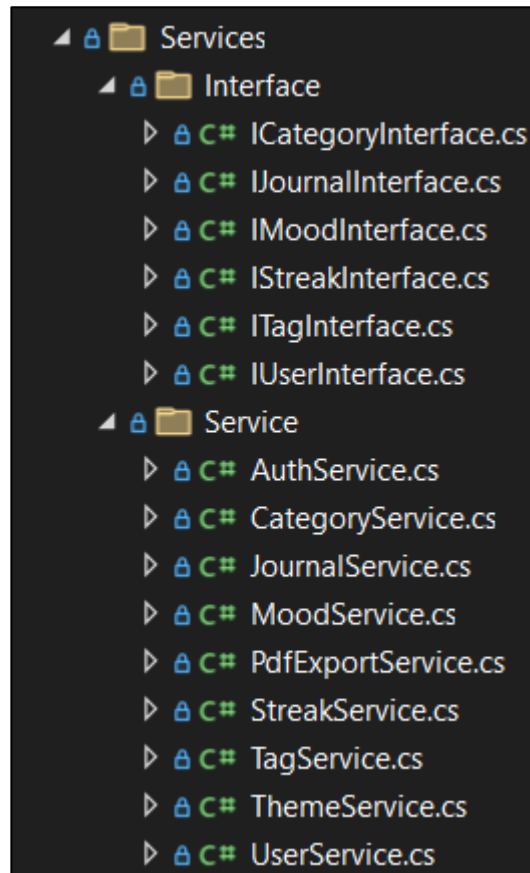
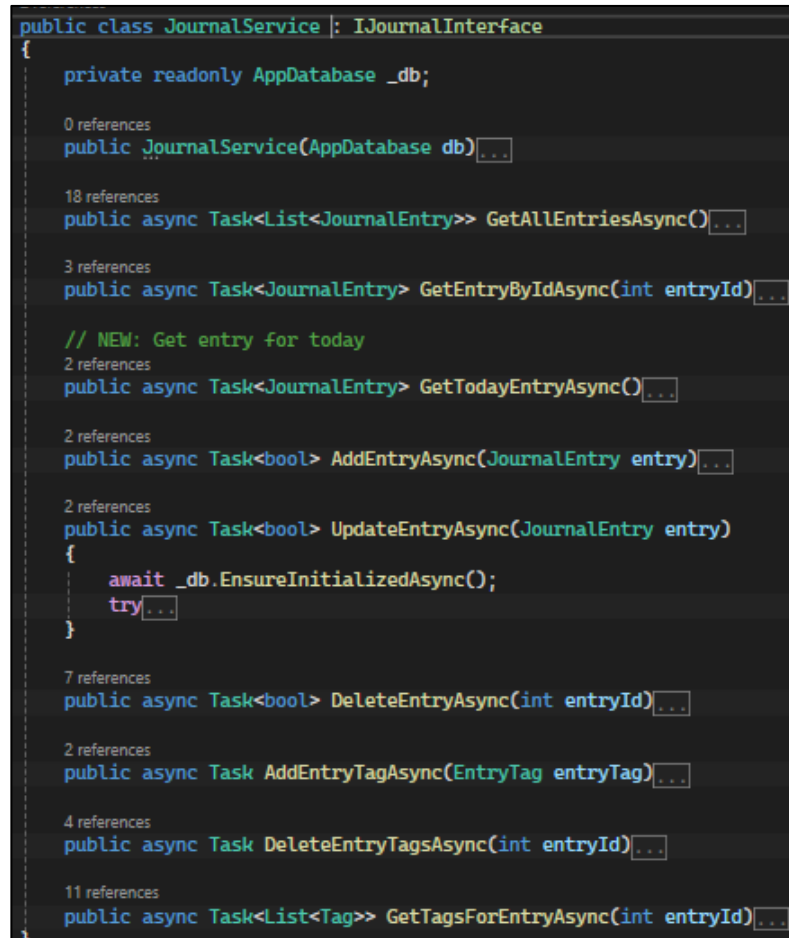


Figure 68: Screenshot showing clear file names

This demonstrates that files are named according to their responsibilities, making it easier to locate and understand different parts of the system. A clear file structure also improves long-term maintainability.

- Descriptive naming conventions

Evidence:



```
public class JournalService : IJournalInterface
{
    private readonly AppDatabase _db;

    0 references
    public JournalService(AppDatabase db) {...}

    18 references
    public async Task<List<JournalEntry>> GetAllEntriesAsync() {...}

    3 references
    public async Task<JournalEntry> GetEntryByIdAsync(int entryId) {...}

    // NEW: Get entry for today
    2 references
    public async Task<JournalEntry> GetTodayEntryAsync() {...}

    2 references
    public async Task<bool> AddEntryAsync(JournalEntry entry) {...}

    2 references
    public async Task<bool> UpdateEntryAsync(JournalEntry entry)
    {
        await _db.EnsureInitializedAsync();
        try {...}
    }

    7 references
    public async Task<bool> DeleteEntryAsync(int entryId) {...}

    2 references
    public async Task AddEntryTagAsync(EntryTag entryTag) {...}

    4 references
    public async Task DeleteEntryTagsAsync(int entryId) {...}

    11 references
    public async Task<List<Tag>> GetTagsForEntryAsync(int entryId) {...}
}
```

Figure 69: Screenshot showing descriptive naming conventions

Using descriptive identifiers helps explain the purpose of variables and methods without relying heavily on comments. This makes the code easier to read and reduces confusion during future updates.

- Consistent formatting and indentation

Evidence:

```

namespace JournalApp1.Helpers
{
    0 references
    public static class MarkdownHelper
    {
        0 references
        public static string WrapSelection(string content, int selectionStart, int selectionLength, string wrapper)
        {
            if (selectionLength == 0)
            {
                // No selection, insert wrapper at cursor
                return content.Insert(selectionStart, wrapper + wrapper);
            }

            var before = content.Substring(0, selectionStart);
            var selected = content.Substring(selectionStart, selectionLength);
            var after = content.Substring(selectionStart + selectionLength);

            return before + wrapper + selected + wrapper + after;
        }

        0 references
        public static string InsertAtCursor(string content, int cursorPosition, string text)
        {
            return content.Insert(cursorPosition, text);
        }

        0 references
        public static string WrapLine(string content, int cursorPosition, string prefix)
        {
            // Find the start of the current line
            var lineStart = content.LastIndexOf('\n', Math.Max(0, cursorPosition - 1)) + 1;

            return content.Insert(lineStart, prefix);
        }
    }
}

```

Figure 70: Maintaining consistent formatting and indentation

Consistent indentation and spacing improve visual clarity and make logical blocks easier to follow. This also ensures uniform coding standards across the entire project.

- Relevant and concise comments

```

// Seed moods safely (no duplicates)
1 reference
private async Task EnsureSeedMoodsAsync()
{
    var existingMoods = await _database.Table<Mood>().ToListAsync();
    var existingNames = existingMoods.Select(m => m.Name).ToHashSet();

    var allMoods = new List<Mood>
    {
        // Positive
        new Mood { Name = "Happy", Type = "Positive" },
        new Mood { Name = "Excited", Type = "Positive" },
        new Mood { Name = "Relaxed", Type = "Positive" },
        new Mood { Name = "Grateful", Type = "Positive" },
        new Mood { Name = "Confident", Type = "Positive" },

        // Neutral
        new Mood { Name = "Calm", Type = "Neutral" },
        new Mood { Name = "Thoughtful", Type = "Neutral" },
        new Mood { Name = "Curious", Type = "Neutral" },
        new Mood { Name = "Nostalgic", Type = "Neutral" },
        new Mood { Name = "Bored", Type = "Neutral" },

        // Negative
        new Mood { Name = "Sad", Type = "Negative" },
        new Mood { Name = "Angry", Type = "Negative" },
        new Mood { Name = "Stressed", Type = "Negative" },
        new Mood { Name = "Lonely", Type = "Negative" },
        new Mood { Name = "Anxious", Type = "Negative" }
    };
}

```

Figure 71: Screenshot showing relevant and concise comments

Comments are used only where necessary to clarify operations. This avoids clutter while still improving understanding of logic.

3.3.2 Code Efficiency

The system was designed to run efficiently by avoiding unnecessary processing and repeated operations. Database queries are kept simple and only retrieve the data that is required at a given time. This approach helps improve performance and ensures the application responds smoothly during use.

Evidence:

- Optimized database query
- Selective data retrieval before seeding

```
var existingCategories = await _database.Table<Category>().ToListAsync();
var existingNames = existingCategories.Select(c => c.Name).ToHashSet();
```

Figure 72: Screenshot showing optimized database query

Existing records are retrieved once and stored in a hash-based collection to allow fast lookups, preventing duplicate inserts and reducing unnecessary database writes.

- Avoidance of redundant inserts

```
if (!existingNames.Contains(mood.Name))
    await _database.InsertAsync(mood);
```

Figure 73: Screenshot showing avoidance of redundant inserts

Conditional insertion logic ensures that only missing records are added, reducing redundant database operations and improving overall efficiency.

- Targeted data retrieval

```
return await _db.Database.Table<Category>().ToListAsync();
```

Figure 74: Screenshots showing targeted data retrieval

Fetching only required data reduces memory usage and improves performance. This approach ensures the application remains responsive as data grows.

- Conditional rendering in the UI

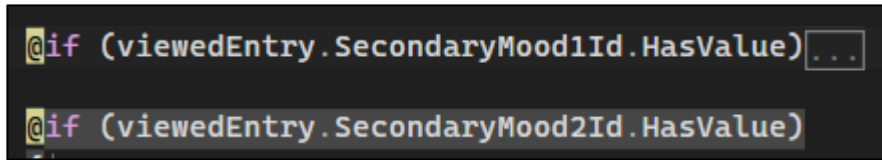


Figure 75: Screenshot showing conditional rendering in UI

Conditional rendering is applied so that secondary mood details appear only when relevant data exists, which helps minimize unnecessary user interface processing.

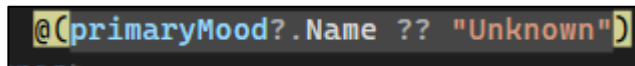


Figure 76: Screenshot showing conditional rendering in UI

Null-safe expressions are implemented to manage missing values smoothly, preventing rendering errors and maintaining a stable and reliable user interface.

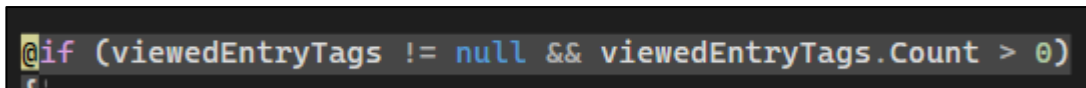


Figure 77: Screenshot showing conditional rendering

It is showing tag information conditionally based on availability

UI elements are rendered only when needed, which reduces unnecessary processing. This contributes to smoother user interaction

3.3.3 Code Modularity

The project follows a modular design by separating different responsibilities into interfaces, services, and UI components. Core functionality is handled within service classes, while user interface files focus only on presentation. Dependency Injection is used to connect components, making the system more flexible and easier to maintain or extend.

- Clear separation of concerns

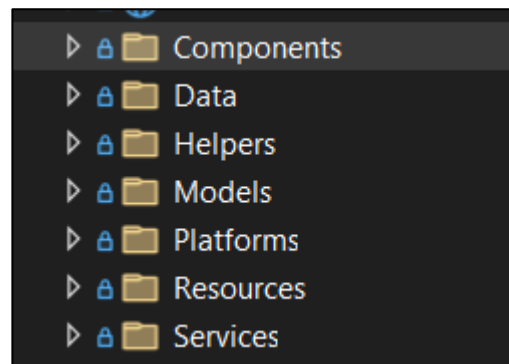


Figure 78: Screenshot of clear separation of concerns

Each layer handles a specific responsibility, making the system easier to understand and modify. This separation also supports scalability and future feature expansion.

- Abstraction

```
@page "/journal"
@using JournalApp1.Models
@using JournalApp1.Services.Interface
@using Markdig
@inject IJournalInterface JournalService
@inject IMoodInterface MoodService
@inject ITagInterface TagService
```

Figure 79: Screenshot of abstraction

- Dependency Injection

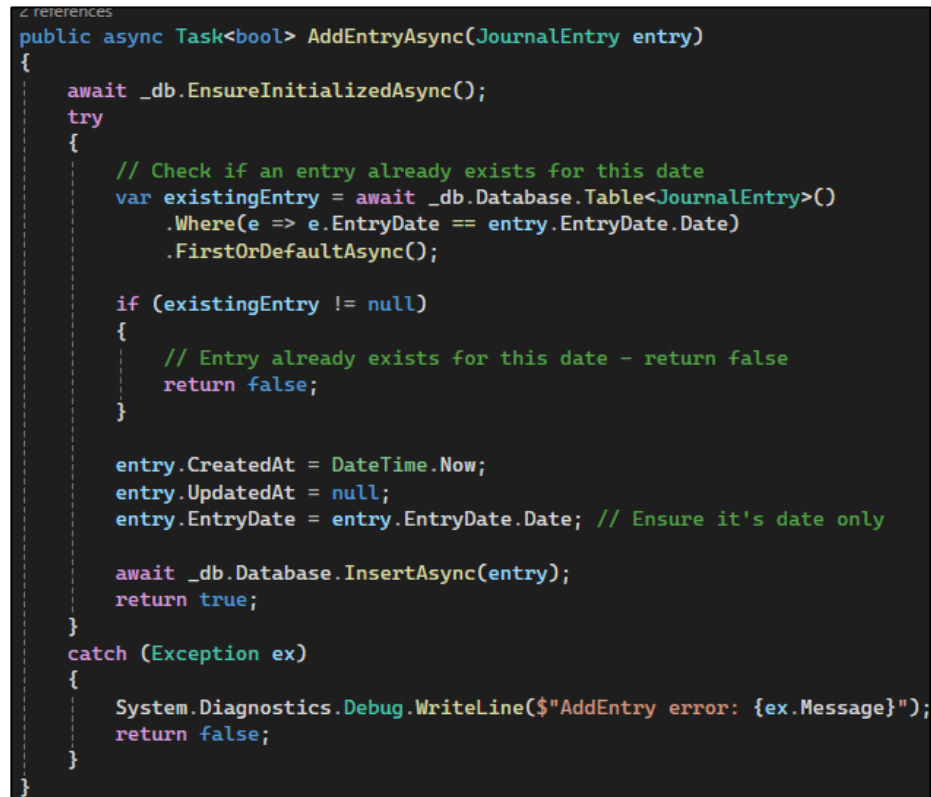
```
2 references
public class JournalService : IJournalInterface
{
    private readonly AppDatabase _db;

    0 references
    public JournalService(AppDatabase db)
    {
        _db = db;
    }
}
```

Figure 80: Screenshot of dependency injection

- Service-based logic implementation

Business logic is handled within services such as JournalService and AuthService and function with logic is created.

A screenshot of a code editor showing the implementation of the AddEntryAsync method in a service. The code is written in C# and includes comments for each step: checking for existing entries, setting timestamps, and inserting the entry into the database. It also features a try-catch block for error handling.

```
z references
public async Task<bool> AddEntryAsync(JournalEntry entry)
{
    await _db.EnsureInitializedAsync();
    try
    {
        // Check if an entry already exists for this date
        var existingEntry = await _db.Database.Table<JournalEntry>()
            .Where(e => e.EntryDate == entry.EntryDate.Date)
            .FirstOrDefaultAsync();

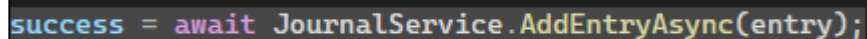
        if (existingEntry != null)
        {
            // Entry already exists for this date - return false
            return false;
        }

        entry.CreatedAt = DateTime.Now;
        entry.UpdatedAt = null;
        entry.EntryDate = entry.EntryDate.Date; // Ensure it's date only

        await _db.Database.InsertAsync(entry);
        return true;
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine($"AddEntry error: {ex.Message}");
        return false;
    }
}
```

Figure 81: Screenshot of business logic in service

Then, the same function is called out by razor files to carry out different functions.

A screenshot of a code editor showing a single line of C# code in a Razor page that calls the AddEntryAsync method of the JournalService.

```
success = await JournalService.AddEntryAsync(entry);
```

Figure 82: Screenshot of service implementation in razor page

Keeping logic out of UI files improves readability and testability. This structure also allows services to be reused by multiple components.

3.3.4 Error handling

Different error-handling techniques were applied to keep the application stable. Input validation, null checks, and exception handling are used to prevent

unexpected crashes. User-friendly messages are shown when errors occur, allowing issues to be identified without exposing technical details.

- **Input validation feedback**

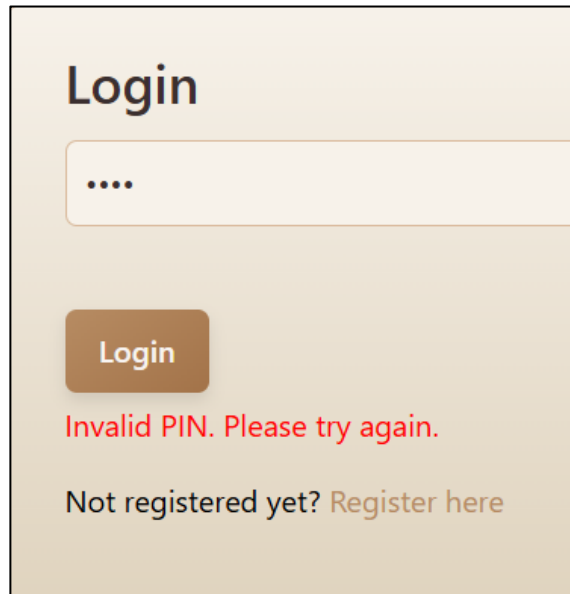


Figure 83: Screenshot of input validation error

Screenshots display validation messages for empty fields or missing required selections. This prevents invalid data from being processed and guides users to correct mistakes. Validation improves both reliability and user experience.

- **Exception handling using try-catch blocks**

```
await _db.EnsureInitializedAsync();
try
{
    entry.UpdatedAt = DateTime.Now;
    entry.EntryDate = entry.EntryDate.Date; // Ensure it's date only

    await _db.Database.UpdateAsync(entry);
    return true;
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine($"UpdateEntry error: {ex.Message}");
    return false;
}
```

Figure 84: Exception handling with try-catch block

Code snippets show database and authentication logic wrapped in try-catch statements.

Exceptions are handled safely without crashing the application. This ensures system stability even when unexpected issues occur.

- **User-friendly error messages**

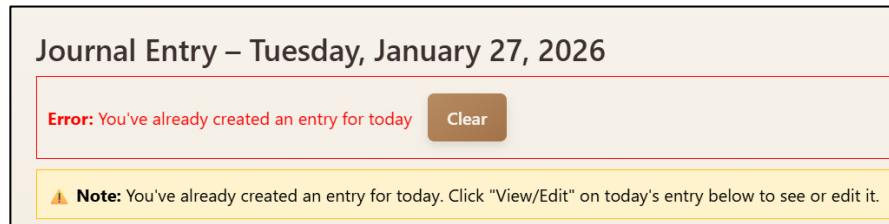


Figure 85: Screenshot of error message

Screenshot show alerts or notifications presented to users during errors. Messages are written in simple language so users understand the issue. Technical details are hidden to maintain security.

- **Authentication error handling**

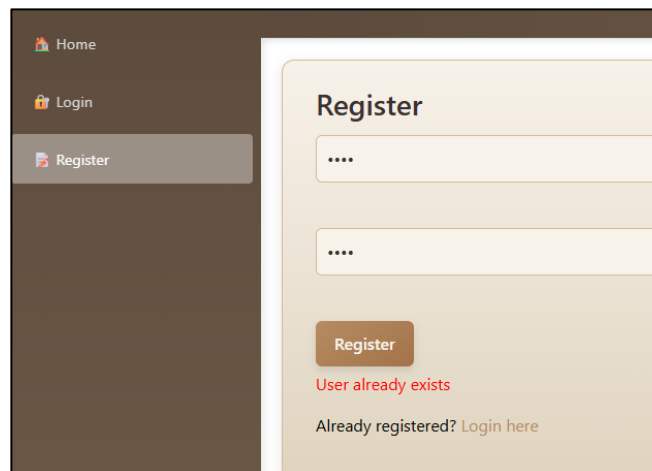


Figure 86: Screenshot of registration authentication error

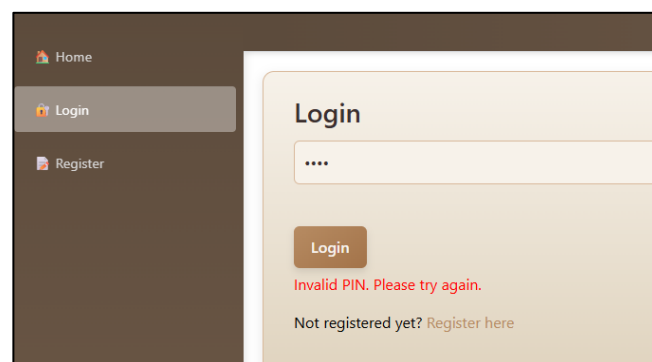


Figure 87: Screenshot of login authentication error

Examples show feedback for incorrect PIN or password attempts. This improves usability while maintaining secure access control. Users are informed without revealing sensitive information.

- **Null and boundary checks**

```
var entry = await GetEntryByIdAsync(entryId);
if (entry == null) return false;
```

Figure 88: Screenshot of null and boundary check code snippet

Code snippet include checks to prevent null reference or out-of-range errors. These checks protect the application from runtime failures. They contribute to a more stable and reliable system.

3.4 Testing

3.4.1 Test 1: Authentication and Access Control

3.4.1.1 Test 1.1: Registration authentication

Test Case:	Attempt to register twice into an account using same pin.
Expected result:	User can register into account with same pin only once.
Actual result:	User doesn't get to register with same pin after registering once.
Conclusion:	The test is successful

Table 3: Test 1.1

Evidence:

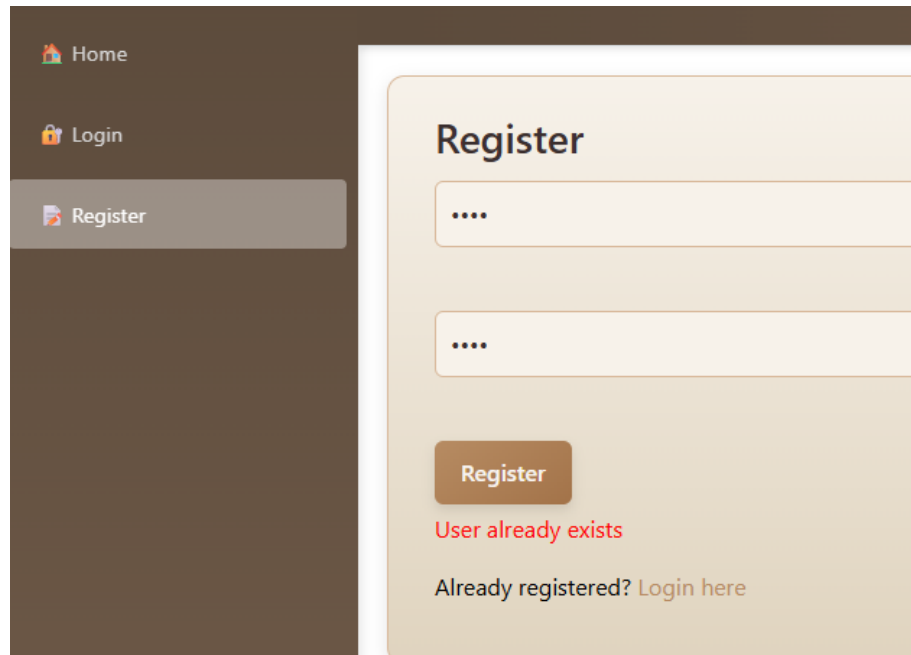


Figure 89: Test 1.1 (Screenshot of registration validation)

3.4.1.2 Test 1.2: Login authentication

Test Case:	Trying to login with pin mismatch after registration.
Expected result:	User cannot login with pin mismatch
Actual result:	User doesn't get to login with pin mismatch.
Conclusion:	The test is successful

Table 4: Test 1.2

Evidence:

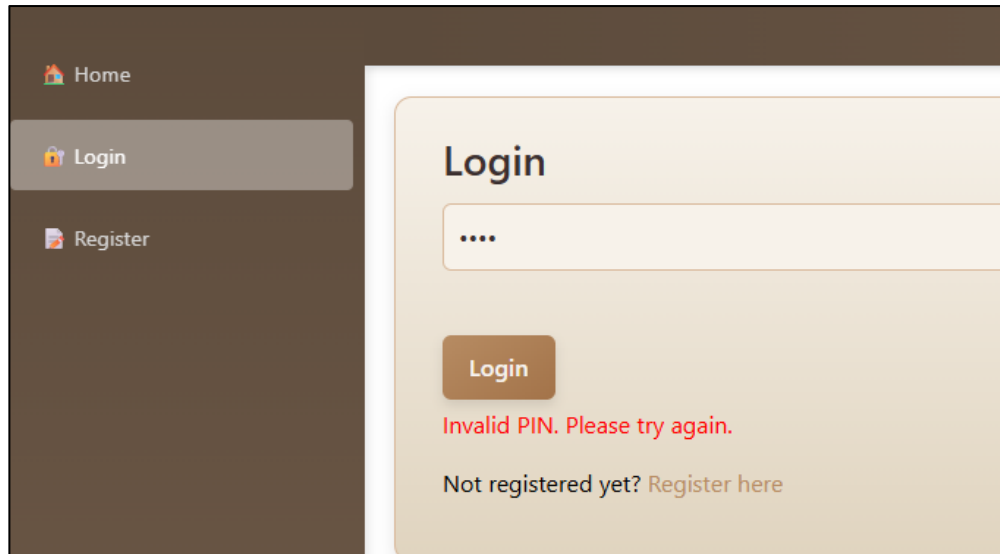


Figure 90: Test 1.2 (Screenshot of login validation):

3.4.2 Test 3: Journal Entry Mood Selection Validation

Test Case:	Select one primary mood and up to two secondary moods for an entry.
Expected result:	The system should require one primary mood and restrict secondary moods to a maximum of two.
Actual result:	Mood selection rules were enforced correctly.
Conclusion:	Mood tracking worked as designed.

Table 5: Test 3

Evidence:

- Screenshot of mood selection with saved values

Primary Mood *
Relaxed (Positive) ▼
Secondary Mood 1 (Optional)
Calm (Neutral) ▼
Secondary Mood 2 (Optional)
Thoughtful (Neutral) ▼

Figure 91: Test 3(Screenshot of mood selection)

Journal Entry – Monday, January 26, 2026

A good day!

Monday, January 26, 2026

Mood: Relaxed (Positive) Calm Thoughtful

Tags: Self-care Finance Weather

So how was today?

- Good
- 1. Very good

Hi there

Hey

Created: 1/26/2026 8:47 PM | Last Updated: Never

Figure 92: Test 3 (Screenshot of saved values at journal after mood selection)

3.4.3 Test 4: Tag Assignment

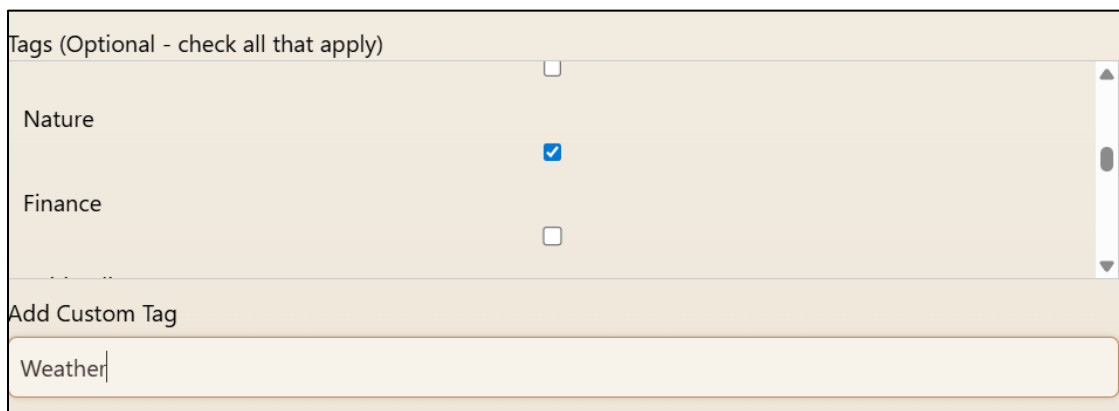
Test Case:	Adding both predefined and custom tags to a journal entry.
------------	--

Expected result:	Tags should be saved and displayed correctly with the entry.
Actual result:	All selected tags were stored and shown accurately.
Conclusion:	Tagging functionality was successful.

Table 6: Test 4

Evidence:

- Screenshot displaying assigned tags



The screenshot shows a web form with a section titled "Tags (Optional - check all that apply)". Below this title is a list of tags: "Nature", "Finance", and "Weather". Each tag has a corresponding checkbox. The "Nature" checkbox is checked, while "Finance" and "Weather" are unchecked. Below the list of tags is a section titled "Add Custom Tag" with a text input field containing the word "Weather".

Figure 93: Test 4 (Screenshot showing chosen tags)

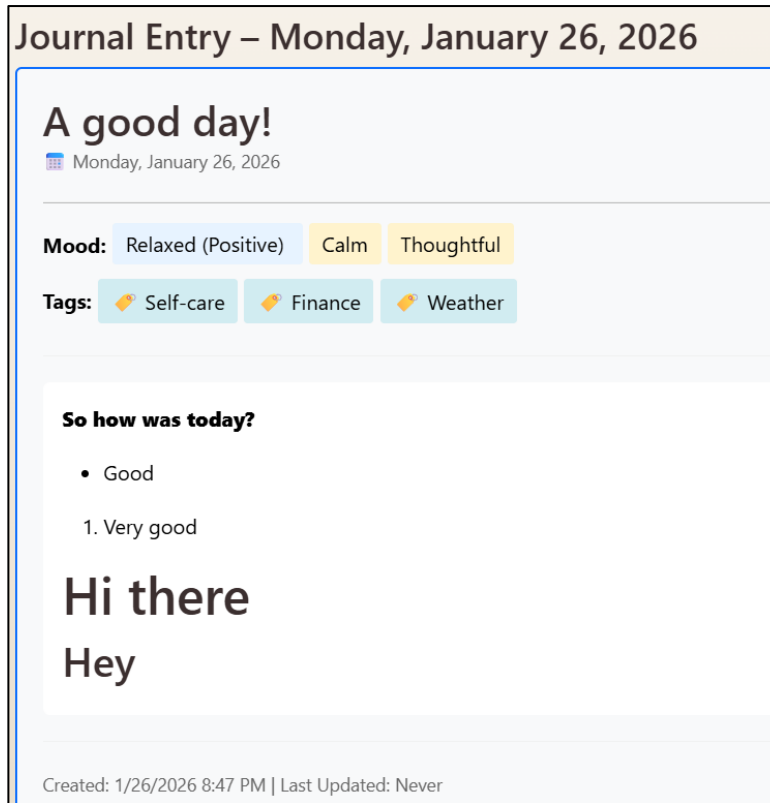


Figure 94: Test 4 (Screenshot displaying assigned tags.)

3.4.4 Test 5: Rich Text and Markdown Formatting

Test Case:	Applying formatting options such as bold text, headings, and lists within a journal entry.
Expected result:	All formatting should remain intact after saving and reopening the entry.
Actual result:	Formatting was preserved and displayed properly.
Conclusion:	Formatting support functioned as intended.

Table 7: Test 5

Evidence:

- Screenshot showing formatted journal content.



Figure 95: Test 5(Screenshot writing formatted journal content)

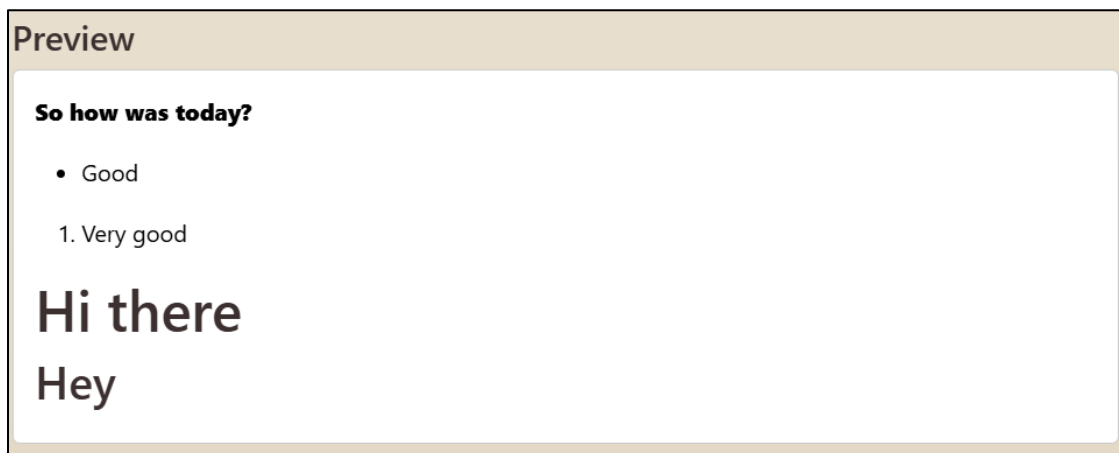


Figure 96: Test 5 (Screenshot showing formatted journal content)

3.4.5 Test 6: Journal Entry CRUD

3.4.5.1 Test 6.1: Creating a Daily Journal Entry

Test Case:	Add a new journal entry for the current date.
------------	---

Expected result:	The system should allow only one entry per day and automatically record the creation time.
Actual result:	The application correctly restricted entry creation to one per day and generated timestamps automatically.
Conclusion:	The feature worked as expected.

Table 8: Test 6.1

Evidence:

Journal Entry – Monday, January 26, 2026

 **Note:** You've already created an entry for today. Click "View/Edit" on today's entry below to see or edit it.

Title

Primary Mood *

Secondary Mood 1 (Optional)

Secondary Mood 2 (Optional)

Tags (Optional - check all that apply)

☐ Projects
 ☐ Planning
 ☐ Reflection

Figure 97: Test 6.1(Screenshot of journal creation)

Past Entries (3)

A good day! Today
 January 26, 2026

Figure 98: Test 6.1(Screenshot showing the newly created entry with its date and timestamp)

3.4.5.2 Test 6.2: Viewing journal entry

Test Case:	View journal entry after creation
Expected result:	User views journal after creation
Actual result:	User sees the journal after creation
Conclusion:	The test is successful

Table 9: Test 6.2

Evidence:

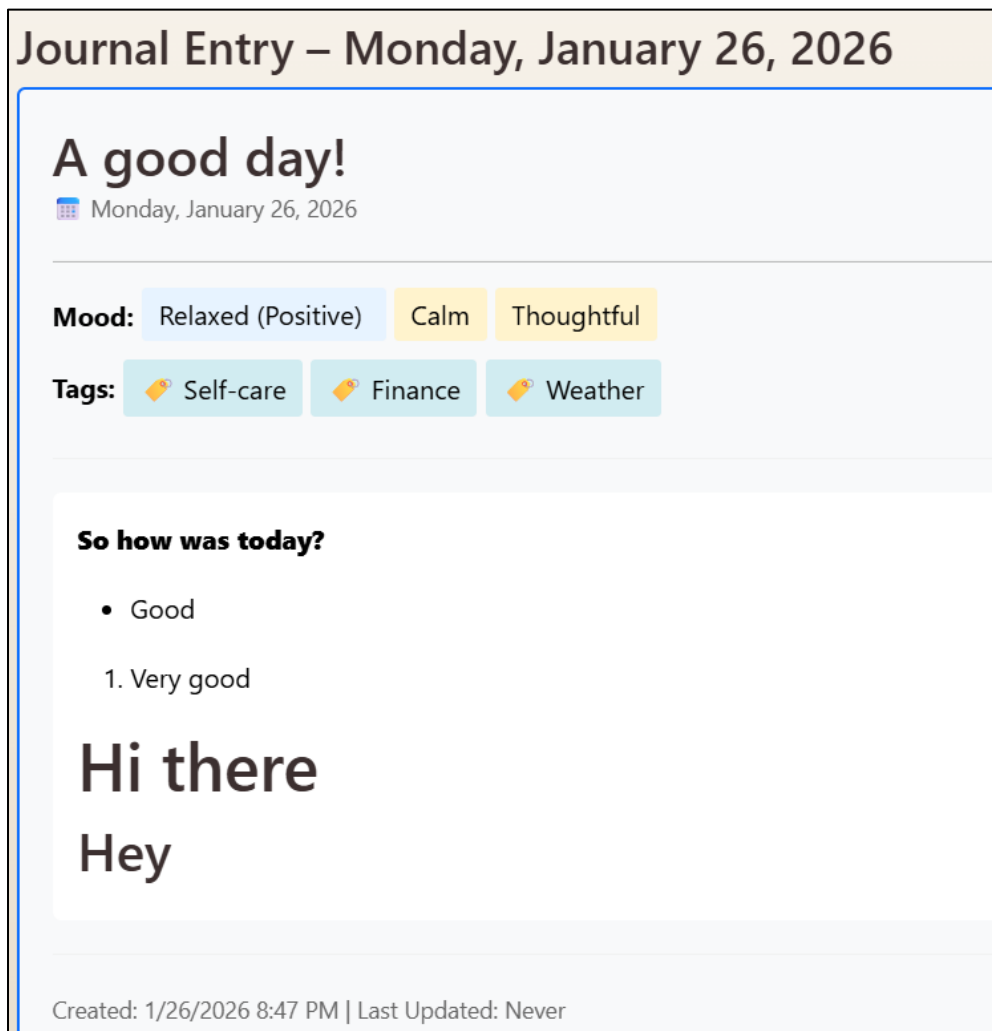


Figure 99: Test 6.2 (Screenshot showing journal entry)

3.4.5.3 Test 6.3: Updating an Existing Entry

Test Case:	Modify the content of an existing journal entry and save the changes.
Expected result:	The updated content should be saved, and the system should refresh the “last updated” timestamp.
Actual result:	The changes were stored correctly, and the updated timestamp was applied automatically.
Conclusion:	The update process was successful.

Table 10: Test 8

Evidence:

Screenshot displaying the updated journal entry and timestamp.

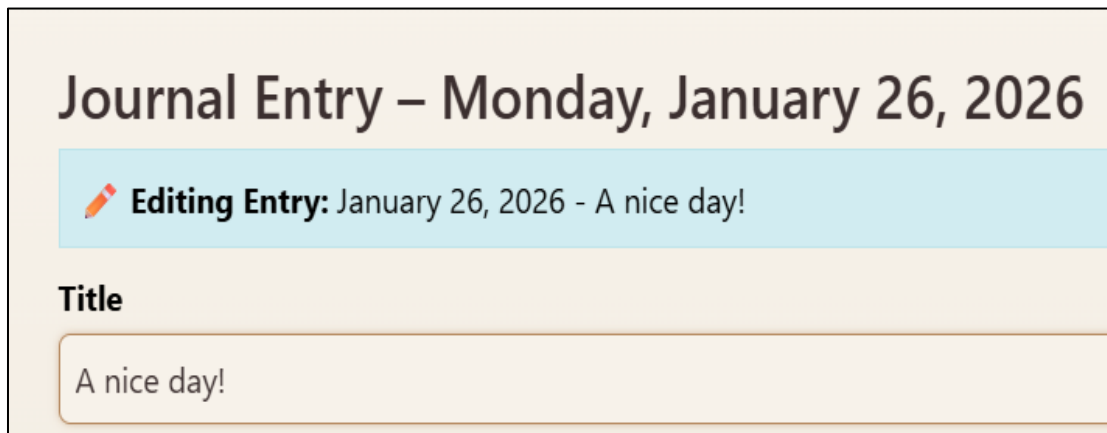


Figure 100: Test 6.3 (Screenshot showing content while updating)

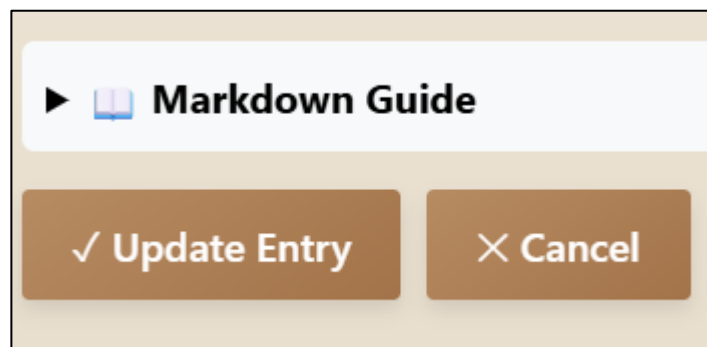


Figure 101: Test 6.3 (Screenshot showing update option)

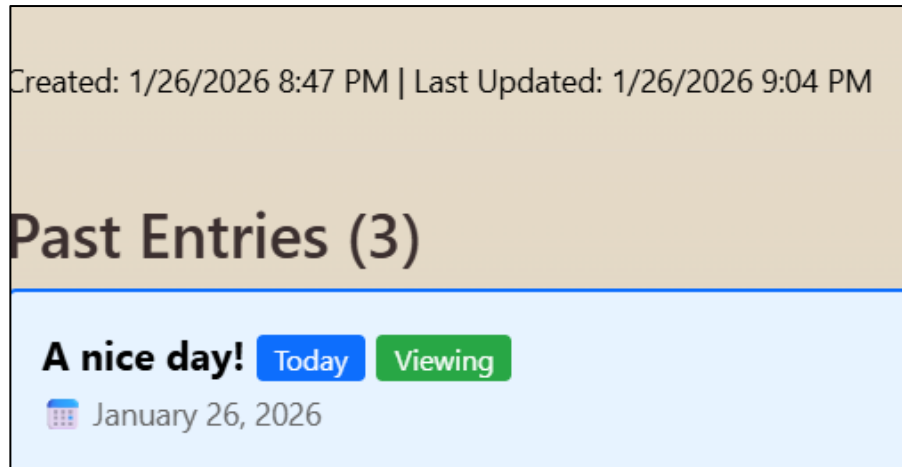


Figure 102: Test 6.3(Screenshot showing updated entry)

3.4.5.4 Test 6.4: Deleting a Journal Entry

Test Case:	Remove a selected journal entry from the system.
Expected result:	The entry should be permanently removed and no longer appear in any view.
Actual result:	The entry was deleted successfully and removed from the journal list and calendar view.
Conclusion:	Deletion functionality performed correctly.

Table 11: Test 6.4

Evidence:

- Screenshot showing the journal list after deletion.

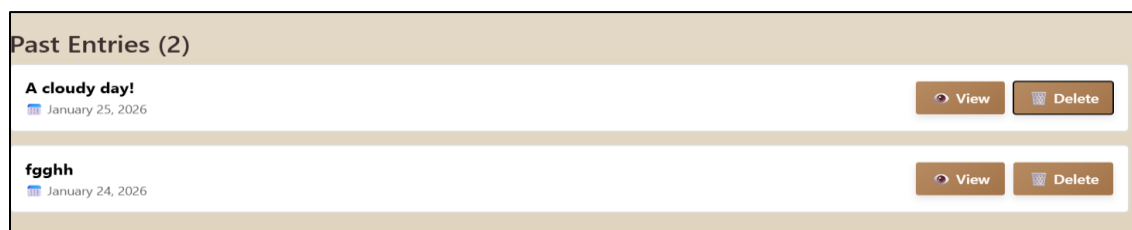


Figure 103: Test 6.4(Screenshot showing journal list after deletion)

- Screenshot showing calendar view after deletion



Figure 104: Test 6.4 evidence

3.4.6 Test 7: Calendar-Based Entry Navigation

Test Case:	Select a date with an existing journal entry from the calendar.
Expected result:	The corresponding journal entry should open correctly.
Actual result:	The correct entry was displayed for the selected date.
Conclusion:	Calendar navigation worked as expected.

Table 12: Test 7

Evidence:

- Screenshot showing the selected date and opened entry.

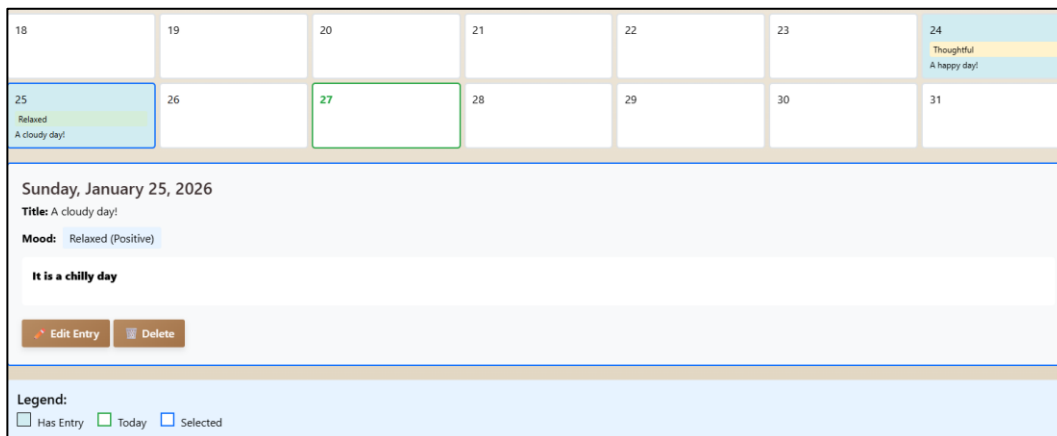


Figure 105: Test 7 (Screenshot showing the selected date and opened entry)

3.4.7 Test 8: Paginated view of past journal entries

Test Case:	Viewing past journal entries in a list
Expected result:	The past journal after journal creation are shown in a list.
Actual result:	Past entries of journal are shown.
Conclusion:	The test is successful

Table 13: Test 8

Evidence:

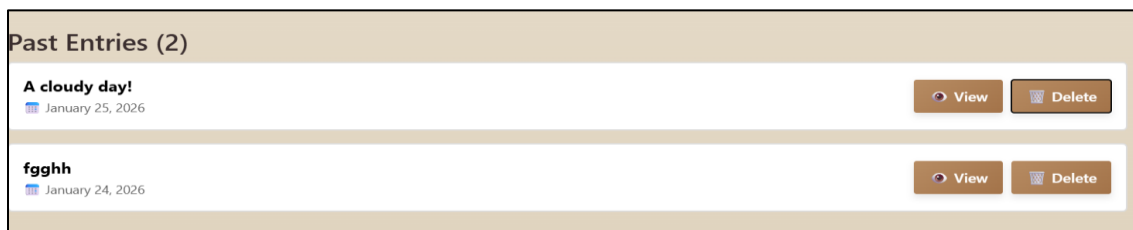


Figure 106: Test 8 (Screenshot of journal entries)

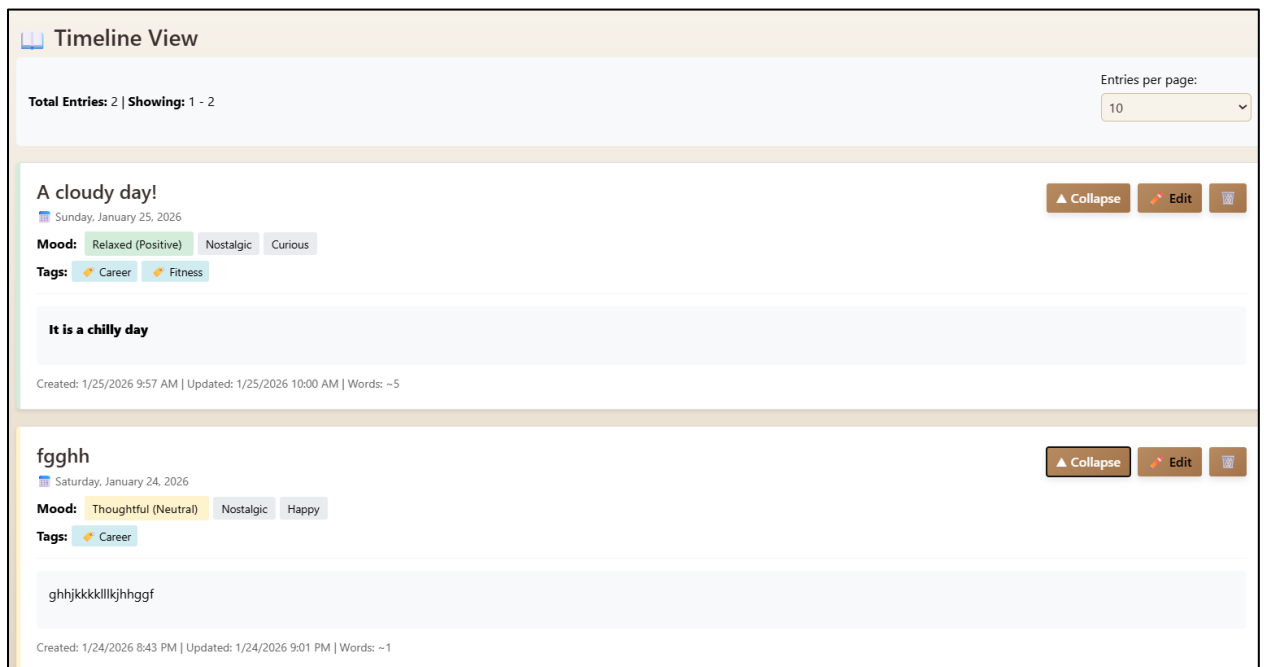


Figure 107: Test 8 (Screenshot of journal entries in a list with expanded content)

3.4.8 Test 9: Search and filter of journals

3.4.8.1 Test 9.1: Search and filter of journals using search bar

Test Case:	Search for entries using keywords in search bar
Expected result:	The system should return only entries that match the selected criteria.
Actual result:	Search and filter results were accurate.
Conclusion:	Search and filtering feature performed correctly.

Table 14: Test 9.1

Evidence:

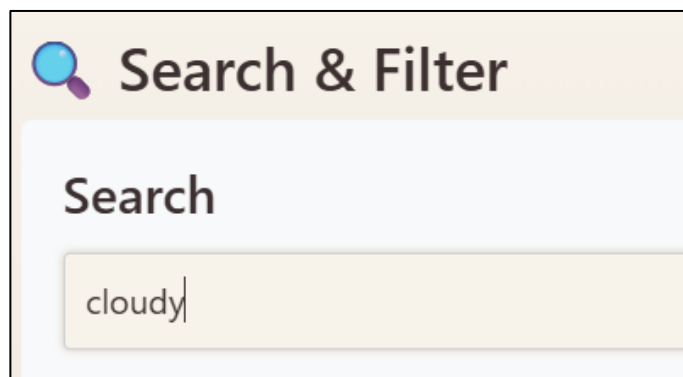


Figure 108: Test 9.1 (Screenshot showing search content)

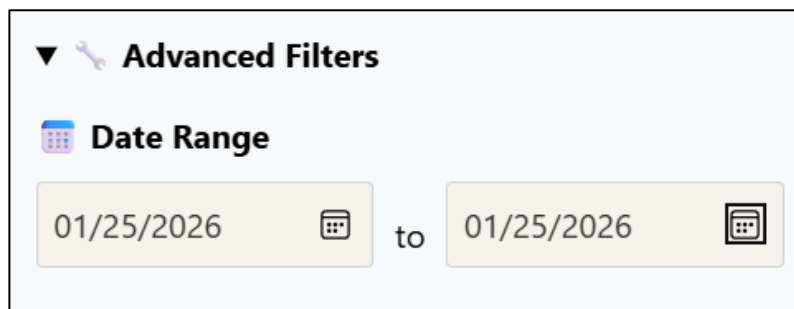


Figure 109: Test 9.1 (Screenshot showing filtered search result)

3.4.8.2 Test 9.2: Search and filter of journals by date

Test Case:	Search for entries by applying date range
Expected result:	The system should return only entries that match the selected criteria
Actual result:	Search and filter results were accurate.
Conclusion:	Search and filtering feature performed correctly.

Table 15: Test 9.2



▼ **Advanced Filters**

Date Range

01/25/2026 to 01/25/2026

Figure 110: Test 9.2 (Screenshot showing filtration content by date)



Results: 1 of 2 entries (Filters active)

A cloudy day!

Sunday, January 25, 2026

Mood: Relaxed (Positive) Nostalgic Curious

Tags: Career Fitness

****It is a chilly day****

Figure 111: Test 9.2 (Screenshot showing filtered search result by date)

3.4.8.3 Test 9.3: Search and filter of journal by mood

Test Case:	Search for entries by applying mood
Expected result:	The system should return only entries that match the selected criteria
Actual result:	Search and filter results were accurate.
Conclusion:	Search and filtering feature performed correctly.

Table 16: Test 9.3


Filter by Mood


Mood Type

Relaxed ▼

Positive ▼

Figure 112: Test 9.3 (Screenshot of filtering by mood)

Results: 1 of 2 entries (Filters active)

A cloudy day!

 Sunday, January 25, 2026

Mood: Relaxed (Positive) Nostalgic Curious

Tags:  Career  Fitness

It is a chilly day

Figure 113: Test 9.3(Screenshot showing filtered search result by mood)

3.4.8.4 Test 9.4: Search and filter of journal by tag

Test Case:	Search for entries by applying date range by applying tag
Expected result:	The system should return only entries that match the selected criteria
Actual result:	Search and filter results were accurate.
Conclusion:	Search and filtering feature performed correctly.

Table 17: Test 9.4

Filter by Tags

All Tags

Work ☒

Career ☐

Figure 114: Test 9.2 (Screenshot of filtering by tag)

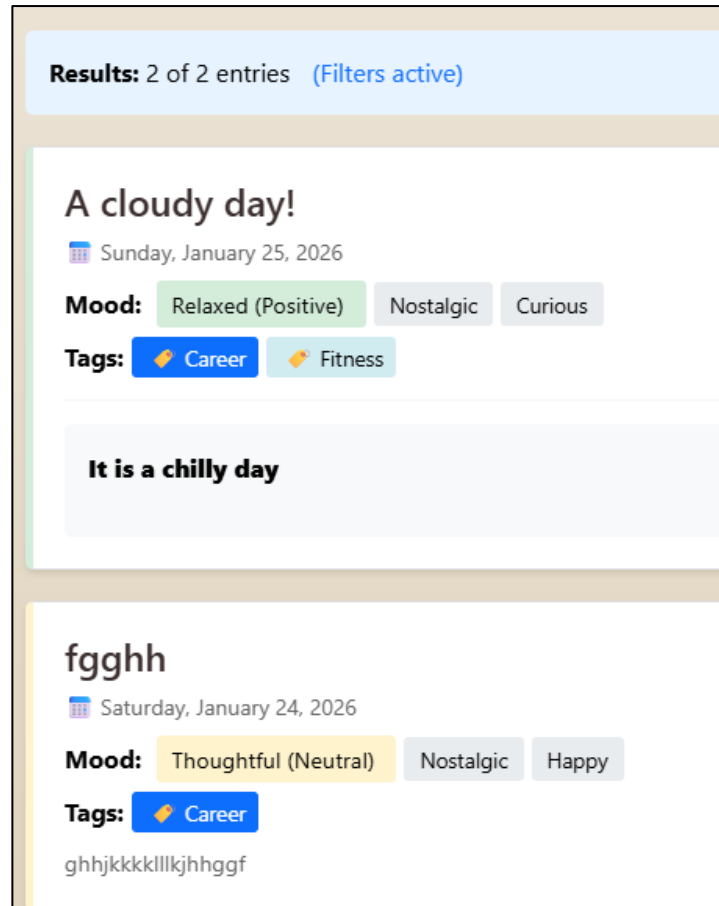


Figure 115: Test 9.4 (Screenshot showing filtered search result by tag)

3.4.9 Test 10: Journaling streak tracking

Test Case:	Create entries over consecutive days and intentionally skip 2 days.
Expected result:	The system should update the current streak, longest streak, and missed days accurately.
Actual result:	Streak values reflected user activity correctly
Conclusion:	Streak tracking functioned properly.

Table 18: Test 10

Evidence:

- Screenshot showing streak details on the dashboard

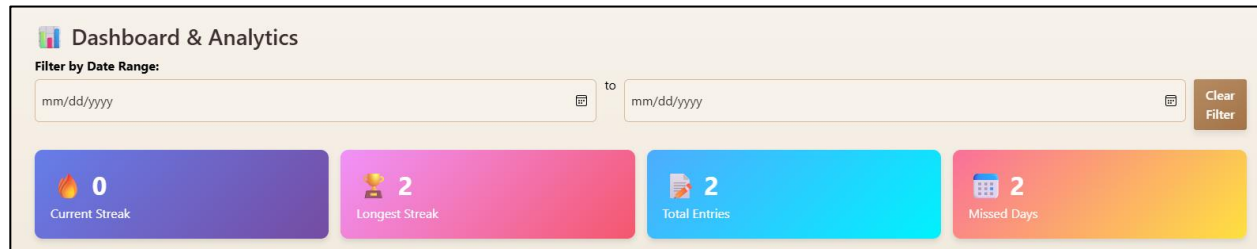


Figure 116: Test 10 (Screenshot showing streak details on the dashboard)

3.4.10 Test 11: Exporting entries as pdf

Test Case:	Export journal entries within a selected date range.
Expected result:	A PDF file containing the selected entries should be generated.
Actual result:	The PDF was created successfully with the correct content.
Conclusion:	Export functionality worked correctly.

Table 19: Test 11

Evidence:

Screenshot showing the export process and generated PDF.

Select Date Range

Start Date:

01/24/2026

End Date:

01/24/2026

 Preview

 Export & Share

Preview: 1 entries will be exported (Jan 24, 2026 - Jan 24, 2026)

Preview

fgggh

 Saturday, January 24, 2026

Mood: Thoughtful (Neutral)

ghhjkkkklllkjhggf

Figure 117: Test 11 (Screenshot showing the preview of journal before pdf export)

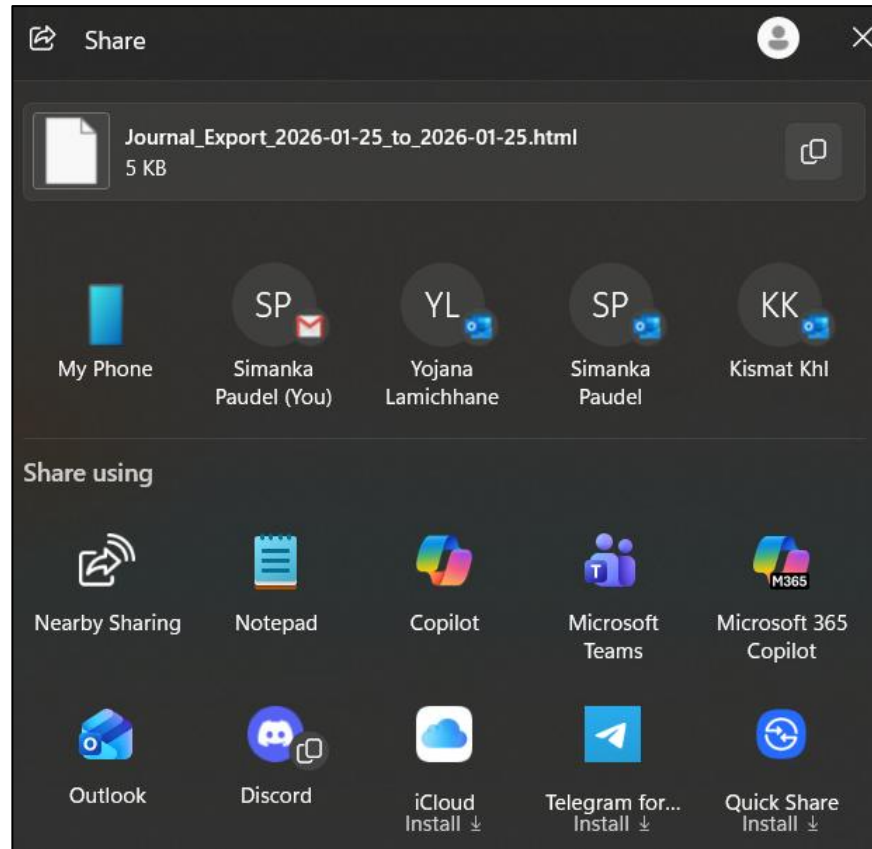


Figure 118: Test 11 (Screenshot showing the export process)



Figure 119: Test 11 (Screenshot showing the generated PDF)

3.4.11 Test 12: Light mode switch

Test Case:	Attempt to switch through light and dark mode
Expected result:	User can switch between light and dark modes
Actual result:	The theme is switched between light and dark
Conclusion:	The test is successful

Table 20: Test 12

Evidence:

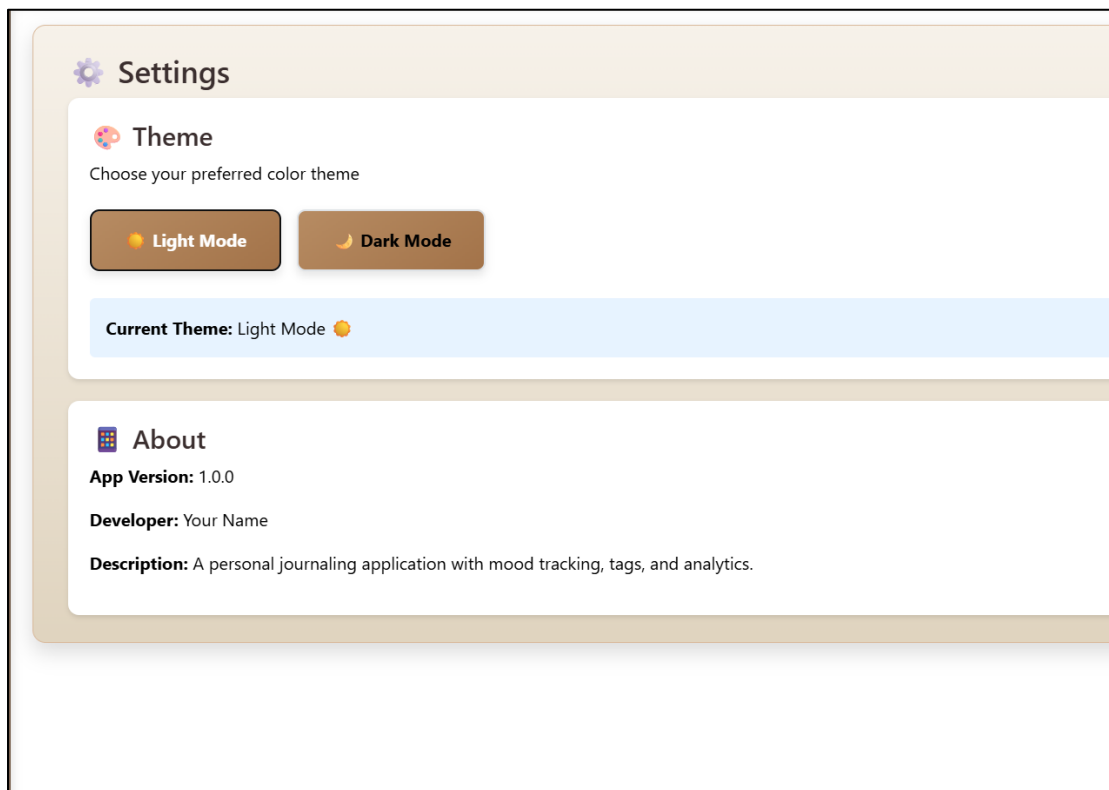


Figure 120: Test 12 (Screenshot of current theme i.e light)

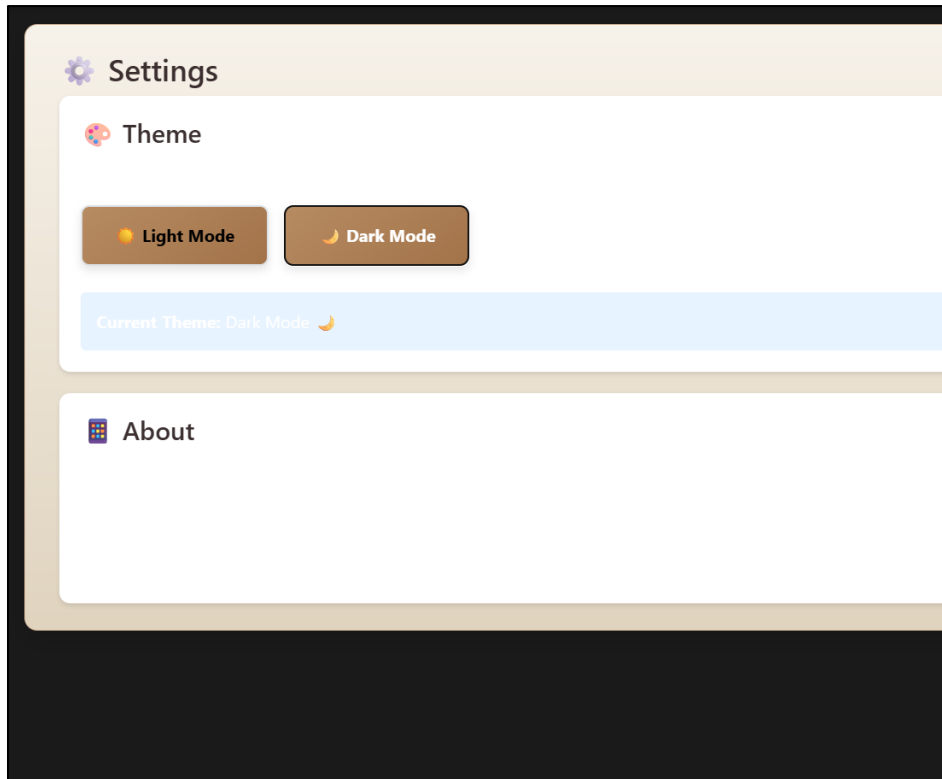


Figure 121: Test 12 (Screenshot of current theme i.e dark)

4 Individual Reflection

This assignment was completed individually, and I handled all phases of the project on my own. My responsibilities covered requirement analysis, system design, application development using C#.NET, database management, feature testing, and compiling the final report.

One of the key challenges was enforcing the restriction of allowing only one journal entry per day while still enabling users to edit or remove their entries when needed. Achieving this required thoughtful implementation at both the database and application layers. Another difficulty involved designing the mood-tracking feature, which included a primary mood along with optional secondary moods. Managing multiple selections while maintaining accurate categorization required careful logic handling.

The analytics component also presented challenges, particularly when transforming stored data into meaningful insights such as mood patterns, entry streaks, and word count summaries. Implementing security features like PIN or password protection helped deepen my understanding of encryption and data protection practices. Additionally, exporting journal entries as PDF files enhanced my experience with file handling and integrating external libraries.

Overall, this project provided valuable hands-on experience in developing a complete desktop application from start to finish. It strengthened my problem-solving abilities, improved my time management skills, and increased my confidence in working with C#.NET and SQLite. The project closely reflected real-world development tasks and contributed significantly to my technical and professional growth.

5 Conclusion

This project successfully met its objective of designing and developing a complete C#.NET-based journaling application that satisfies both functional and non-functional requirements. The system enables users to create and manage daily journal entries using rich-text/Markdown formatting while supporting mood tracking, tagging, calendar navigation, analytics, and secure access. The restriction of one journal entry per day was carefully implemented to maintain data consistency and support accurate streak tracking.

The application was developed using a service-oriented architecture with clear separation of concerns, abstraction, and dependency injection. These design choices enhanced code readability, modularity, maintainability, and reusability. Robust error handling, input validation, and user-friendly feedback mechanisms contributed to system stability and a reliable user experience. Version control practices were followed throughout development to ensure organized progress tracking and effective change management.

Despite meeting all required objectives, there are several opportunities for future enhancement. The application could be extended with cloud-based synchronization to allow data access across multiple devices. Additional analytics features, such as advanced mood correlation insights and customizable reports, could provide deeper personal reflections for users. Cross-platform support for mobile and web environments would improve accessibility, while further enhancements to security—such as biometric authentication or encrypted backups—could strengthen data protection. Performance optimizations and UI refinements could also be introduced as the user base grows.

Overall, this project demonstrates a strong application of software engineering principles and provides a solid foundation for future development. The experience gained from this project reflects real-world development practices and prepares the system for further scalability and enhancement.

5.1 Future Work

This project successfully met its goal of designing and implementing a complete C#.NET journaling application that satisfies both the functional and non-functional requirements defined in the given scenario. The developed system offers users a well-organized yet flexible environment to create daily journal entries using rich-text or Markdown formatting, record moods, assign tags, and review personal patterns through analytical features.

All essential functionalities were implemented as intended, including daily journal entry creation with system-generated timestamps, primary and secondary mood tracking, tagging support, calendar-based navigation, search and filtering options, streak monitoring, analytical dashboards, security features, and PDF export capabilities. Particular care was taken to enforce the constraint of allowing only one journal entry per day, ensuring data accuracy and reliable streak tracking.

From a development standpoint, the application follows sound software engineering principles such as service-oriented design, abstraction, dependency injection, and clear separation of concerns. These practices contributed to improved code clarity, modular structure, ease of maintenance, and better testability. Robust error handling and input validation mechanisms were also applied to enhance system stability and provide a smooth user experience. Version control tools were consistently used throughout the development process to manage changes and maintain an organized development history.

Although the system fully meets the project requirements, there is potential for further enhancement. Possible future improvements include cloud-based data synchronization, multi-platform support, more advanced data visualization for analytics, and intelligent mood analysis features. Overall, this project demonstrates the effective application of C#.NET development concepts in a practical context and reflects a strong understanding of software design, implementation, and documentation best practices.

